

A Modular Framework for Stack-Heap and Value Abstractions

Giacomo Boldini^[0009-0006-4741-0033], Luca Negrini^{*[0000-0001-9930-8854]}, Luca Olivieri^[0000-0001-8074-8980], and Pietro Ferrara^[0000-0002-4678-933X]

Ca' Foscari University of Venice, Venice, Italy

{giacomo.boldini, luca.negrini, luca.olivieri, pietro.ferrara}@unive.it

* Corresponding author

Abstract. Advanced static program analysis requires reasoning on the semantics of non-trivial program behaviors (e.g., pointers and complex data structures such as lists and sets, functions, and objects) and how they affect the memory. In most programming languages, static and dynamic allocations are typically managed by the stack and the heap, respectively. However, how allocations behave and how the memory is managed at runtime can vary significantly depending on the programming language being analyzed. Proper handling of these aspects is essential, as an accurate memory model enables the detection of critical issues such as buffer overflows and underflows, use-after-free errors, and null pointer exceptions prior to execution, that is, before such erroneous behaviors occur.

In this paper, we propose and formalize a generic memory framework to handle stack and heap memory during the analysis, that is able to support various behaviors from different programming languages (e.g., C, C++, Java, and Python), while remaining parametric, allowing different memory and value analyses to be independently chosen and combined. It relies on the Abstract Interpretation theory and enables sound approximation of different memory models and program behaviors. We introduce a *split state* abstraction that separates value and memory analyses into two modular abstract domains. These domains interact through a set of *memory identifiers*, along with a set of operations defined by the domains to manipulate them, allowing the framework to capture both value information and structural memory relationships.

Keywords: Abstract interpretation · Static analysis · Memory handling · Heap analysis · Value analysis · Program semantics.

1 Introduction

During execution, a program uses memory to store and retrieve data. This memory is typically split into the *stack* and *heap*. Stack memory is used for *static* allocation with a bounded size, such as local variables within functions. In contrast, heap memory handles *dynamic* allocation, making it suitable for data and

objects whose size can change at runtime or that need to persist beyond the lifetime of a single function call. Both kinds of memory can contain several types of values: from so-called primitive types such as integers or floating point numbers, to arbitrarily complex data structures such as lists, sets, functions, and objects. Depending on the programming language, memory management can be completely different. In languages like C or C++, developers have complete control over memory: allocations and deallocations are manual, and it is the developer’s responsibility to ensure that memory is used correctly (e.g., that a buffer is only accessed within its allocated bounds, or that memory is deallocated when it is no longer needed). In contrast, languages like Java or Python use automatic memory management, where the runtime environment handles memory allocation and deallocation, often through a garbage collection mechanism.

In static program analysis, precisely approximating the memory is crucial. An accurate memory model enables detection of issues such as buffer overflows or underflows, use-after-free, or null pointer dereferences. Some of these issues also require a precise analysis of the values stored in memory, such as the size of a buffer or the address of a pointer. Conversely, properties targeting the values handled by the program, such as the absence of overflows or arithmetic errors, require a precise analysis of the memory layout of the program to retrieve the necessary information about values stored in memory.

Despite this close relation, the fields of memory analyses and value analyses have developed orthogonally to one another, with several options to pick for both [1,38,27,32,31,17]. Any value analysis can be combined with any memory analysis, and it is well-known that such a combination is required for any meaningful program analysis [39].

Over the past decade, two main approaches have dominated the static analysis of programs: (i) analyzers that focus on value information, which first preprocess the program using a dedicated heap analysis and then replace heap accesses with symbolic variables for example Clousot [18], and (ii) heap abstraction techniques for example TVLA [28] that either do not track value information or require manual extensions such as adding specific predicates to capture particular value properties. While recent work (see Section 2) has advanced the integration of heap and value analyses, truly generic and fully automatic analyzers, that is, approaches not bound to a specific heap or value domain and not requiring user provided annotations, are still limited.

Motivating Example. Before introducing the formal framework, we present a motivating example that highlights common challenges in heap memory analysis, including dynamic allocations, pointer aliasing, and complex structures. Consider the code snippet written in C of Listing 1, coming from the SV-COMP benchmark¹. The code repeatedly allocates heap nodes around a stack-allocated object `list`. Two pointers, `x` and `tail`, are both initialised to `&list` (lines 7 and 8), suggesting that both are intended to track the evolving structure. One would

¹ <https://github.com/sosy-lab/sv-benchmarks/blob/master/c/heap-manipulation/dancing.c> Accessed (05/2026)

expect both to be updated as the data structure grows. In practice, however, only `x` is ever updated (line 21), while `tail` is never reassigned, `tail->R = n` always overwrites `list.R` with the address of the latest node (line 18). Specifically, after k iterations `list.R` points to the k -th node, while all earlier nodes (except possibly the one pointed to by `x`) are unreachable. Similarly, `n->L = tail` sets every node's `L` field to `&list` (line 16), so each heap node independently points back to `list`. The pointer `x` may therefore alias `list` or any heap node allocated during any previous iteration.

Verifying this kind of issues is non-trivial in program static analysis. Capturing such behaviours demands accurate stack and heap management, and without it an analysis can easily miss lost references or unintended aliasing. At the same time, it requires abstractions, but excessive over-approximation may render the analysis impractically imprecise, obscuring the very behaviours it aims to capture and thereby weakening its usefulness in practice.

Listing 1. C slice from `dancing.c`.

```

1  struct node { struct node *L; struct node *R; };
2
3  struct node list;
4  list.L=0;
5  list.R=0;
6
7  struct node *x=&list;
8  struct node *tail=&list;
9
10 while(__VERIFIER_nondet_bool())
11 {
12     struct node *n=malloc(sizeof(struct node));
13     if(n==0)
14         break;
15
16     n->L=tail;
17     n->R=0;
18     tail->R=n;
19
20     if(__VERIFIER_nondet_bool())
21         x=n;
22 }
```

Contributions. This work presents a modular framework for the combined static analysis of stack and heap memory, formalised over μLL , a small imperative language extending μJS [3] with explicit pointers, address-of, dereference, field access, and heap allocation. Our main technical contributions are as follows:

- A formal operational semantics for μLL modelling both stack and heap, with support to novel language features introduced.

- A split state adapting the construction of whole value analysis proposed in Negrini [35]. We reformulate the concrete state as a Galois isomorphism separating value and address information with no loss of precision.
- A parametric abstract framework in which a value domain and a memory domain cooperate via a lightweight interface and a substitution mechanism, inspired by generic object-oriented memory frameworks [20,19]. Whereas those works target object-oriented languages with a single heap, our framework supports a broader range of memory models, including languages such as C, where diverse forms of memory manipulation are possible and execution is not constrained by object-oriented abstractions.
- A framework instantiation with non-relational numerical domains [32] and Andersen-style points-to analysis [1].

Paper Structure. The structure of the paper is as follows. Section 2 deals with related work. Section 3 provides the necessary background information. Section 4 introduces the μ LL language and its semantics. Section 5 and Section 6 present our main contribution defining and formalizing the split state and abstract state. Section 7 demonstrates the modularity and practical applicability of the proposed framework by instantiating it with concrete abstract domains. Section 8 concludes the paper. All proofs, together with some additional definitions, are omitted from this paper; they are available in the version containing the technical report [9].

2 Related Work

In this section, we provide a brief overview of existing frameworks and tools for memory management. The literature can be divided into approaches tailored to specific memory problems and more general-purpose frameworks. For instance, Gopan et al. [22,23] propose a solution specialized for numerical analysis and array operations, which inherently require reasoning about both heap and value domains, since arrays combine memory structure with indexes and content-dependent properties. However, these kinds of problems are orthogonal to our study, since we focused on how to combine heap and value analyses for more general purposes, data structures, and memory models.

The broader difficulty of reasoning about memory and pointers has long been recognized. As early as 2001, Hind [25] observed that, despite the substantial body of research available at the time, memory management and pointer-related issues remained complex and challenging problems. This observation continues to motivate subsequent work in the area. In particular, Kanvar et al. [27] provide a systematic taxonomy of heap memory models used in static analysis, classifying them as storeless, store-based, and hybrid, and surveying summarization techniques such as k-limiting, allocation-site based, pattern-based, variable-based, and other instrumentation predicates, as well as higher-order logic approaches.

About abstract interpretation approaches, to the best of our knowledge, only a limited number of prior works have addressed a general integration of heap and

value management. Chang and Rival [11] proposed an abstract domain that integrates data abstraction with user-provided specifications of data structures. In contrast, our approach is based on heap identifiers and does not require explicit specifications of data structures. They also introduced a modular integration of shape and numerical abstract domains [12], where shape analysis is based on points-to predicates, and the numerical domain tracks information using a symbolic representation of data stored in the heap. This differs slightly from our notion of memory identifiers, which are designed to abstract memory locations. Moreover, their approach focuses on shape analyses based on summarization and materialization of nodes. As a result, when a node is materialized, the analysis requires to maintain a disjunctive abstraction represented by a set of possible shapes. Blazy et al. [8] present an *abstract memory functor* inspired by the memory model of C programs. Basically, they design an abstract memory domain that is able to handle points-to information and it is parameterized by a numerical abstract domain. Then, they implemented it into a formally verified static analyzer that is not dealing precisely with memory [7]. Their evaluation considers both non-relational (intervals [16]) and relational (polyhedra [17]) numerical abstract domains. Miné [30] proposes a memory abstraction that translates the semantics of all memory accesses into a semantics over a set of scalar variables. In this abstraction, a cell represents a memory region holding a scalar value together with its type. The approach is parameterized by a numerical domain, but it does not support summary nodes or dynamic allocation. It was initially implemented in ASTRÉE [6], and has more recently been adopted and extended in MOPSA [26]. To handle pointer dereferences, MOPSA builds on the cell domain, and relies on recency abstractions [5] to model dynamic memory allocation. For each allocation site, recency abstraction distinguishes the most recently allocated block from all previous ones, which are summarized into a single weak memory block. Allocation sites are configurable [34] and are typically defined by program locations. However, this summarization can reduce precision, then alternative abstractions are required to preserve separation between memory blocks during loop unrolling, as in the analysis of C programs [33]. Milanese and Miné [29] also proposed an under-approximation backward analysis for the memory model of MOPSA to enhance both precision and efficiency in the analysis of C programs. Recency abstraction has also been applied for other languages and specific contexts. For instance, Calzavara et al. [10] leverage ideas from recency abstraction to design a sound, flow-sensitive heap abstraction for the static analysis of Android applications written in Java language.

Finally, our work builds upon Ferrara [20,19] and Negrini [35]. The main idea of [20,19] is to introduce an intermediate abstraction for heap and value analyses, by defining a split state, which rewrites the concrete state (i.e., program memory and concrete semantics) to separate values of different data types into distinct maps. The heap analysis aims to abstract the heap structure, while the value domain abstracts the values of non-reference variables, as well as heap locations. Intuitively, the heap analysis approximates concrete locations through heap identifiers, while the value analysis tracks information on these identifiers.

This provides several advantages: (i) a more convenient state representation that simplifies definitions and proofs, (ii) a modular design where existing abstract domains can be applied independently to each sub-memory, reusing standard operations with minimal changes, and (iii) semantic preservation, since proving soundness/completeness is equivalent between the split and concrete states. We extend this setting by widening the set of supported operations to arbitrary references and dereferences, and to dynamic field accesses. This broadens the scope of the framework beyond object-oriented languages. Moreover, we improve on its formalization and soundness proofs borrowing the isomorphism approach from [35]. In particular, instead of building a Galois Connection between the concrete and split states, followed by a second Galois Connection between the split and abstract states, we enforce a Galois Isomorphism between the concrete and split states, ensuring that no loss in precision occurs when moving from the concrete to the split state. This is a key weakness of the original framework, that instead has two possible sources of imprecision.

3 Background

Sets. A set X is a (possibly infinite) collection of elements. We write $x \in X$ for membership, $\emptyset$ for the empty set, $|X|$ for cardinality, and $\wp(X)$ for the powerset. Given sets X and Y , we use standard notation: $X \subseteq Y$ for subset, $X \cup Y$ for union, $X \cap Y$ for intersection, $X \setminus Y$ for set difference, and $X \times Y = \{(x, y) \mid x \in X \wedge y \in Y\}$ for the Cartesian product. Sets can be defined extensionally, e.g., $\{x_0, x_1, \dots\}$, or in terms of a predicate, e.g., $\{x \mid \phi(x)\}$, meaning all x such that a predicate $\phi(x)$ holds.

Functions. A function $f : X \rightarrow Y$ is a subset of the Cartesian product $X \times Y$ such that $\nexists (x, y), (z, w) \in f : x = z \wedge y \neq w$, meaning that each $x \in X$ is associated with at most one $y \in Y$. The set X is called *domain*, denoted as $\text{dom}(f)$, the set Y is called *co-domain*, denoted as $\text{codom}(f)$. The identity function is denoted by id . Similarly to sets, functions can be defined extensionally, e.g., $\{(x_0, y_0), (x_1, y_1), \dots\}$, or in terms of a predicate, e.g., $\{(x, y) \mid \phi(x, y)\}$, meaning all pairs (x, y) such that a predicate $\phi(x, y)$ holds. Given a function f and two elements x and y , the update operator $f[x \mapsto y]$ denotes $f \setminus \{(x, f(x))\} \cup \{(x, y)\}$ when $x \in \text{dom}(f)$, and $f \cup \{(x, y)\}$ otherwise. Abusing notation, we also use chained updates $f[x_1 \mapsto y_1, \dots, x_n \mapsto y_n]$, which denotes successive application of the function update, i.e., $f^0 = f$, $f^i = f^{i-1}[x_i \mapsto y_i]$, whose result is f^n .

Ordered Structures. A set X with a partial ordering relation $\sqsubseteq_X \subseteq X \times X$ is a poset, denoted by $\langle X, \sqsubseteq_X \rangle$. If a poset has a bottom element $\perp_X$ and is closed under finitary applications of the least upper bound ($\text{lub}, \sqcup_X$) operator of X , it is called a complete partial order (cpo), denoted as $\langle X, \sqsubseteq_X, \sqcup_X, \perp_X \rangle$. Moreover, a lattice $\langle X, \sqsubseteq_X, \sqcup_X, \sqcap_X \rangle$ is a poset having a minimum element (bottom, $\perp_X \in X$), a maximum element (top, $\top_X \in X$) and closed under finitary applications of the least upper bound ($\text{lub}, \sqcup_X$) and the greatest lower

bound ($\text{glb}, \sqcap_X$) operators. A *complete* lattice is closed under arbitrary lub and glb, so that $\bigsqcup Y \in X$ and $\bigsqcap Y \in X$, for all $Y \subseteq X$, and it is denoted as $\langle X, \sqsubseteq_X, \sqcup_X, \sqcap_X, \top_X, \perp_X \rangle$. Provided there is no ambiguity, we will omit subscripts of each operator for clarity. Complete lattices can be derived from other structures. For instance, given a set X , $\langle \wp(X), \subseteq, \cup, \cap, X, \emptyset \rangle$ is a complete lattice since $\subseteq$ is a partial ordering relation, $\cup$ and $\cap$ are closed w.r.t. $\wp(X)$, and $\forall Y \in \wp(X) : \emptyset \subseteq Y \subseteq X$. By duality, $\langle \wp(X), \supseteq, \cap, \cup, \emptyset, X \rangle$ is also complete. Moreover, given $\langle X, \sqsubseteq, \sqcup, \sqcap, \top, \perp \rangle$ and a set Y , the *functional lift* [14] of X w.r.t. Y is the complete lattice $\langle Y \rightarrow X, \sqsubseteq, \sqcup, \sqcap, \perp, \top \rangle$ of total functions $Y \rightarrow X$, that is, of functions defined on all elements of Y . Lattice operators are defined as point-wise applications of operators over X on all $y \in Y$. Lastly, given a finite set of complete lattices $\langle Y_i, \sqsubseteq_{Y_i}, \sqcup_{Y_i}, \sqcap_{Y_i}, \perp_{Y_i}, \top_{Y_i} \rangle, i \in \Delta \subset \mathbb{N}$, their Cartesian product [15] is the complete lattice $\langle \times_{i \in \Delta} Y_i, \sqsubseteq, \sqcup, \sqcap, \perp, \top \rangle$, where lattice operators are component-wise applications of the operators over each Y_i . Given a poset $\langle X, \sqsubseteq_X \rangle$, an increasing chain $C \subseteq X$ is a possibly infinite sequence of elements $x_0, x_1, \dots$ of X such that $x_0 \sqsubseteq_X x_1 \sqsubseteq_X \dots$.

Abstract Interpretation. Abstract Interpretation [16,15] is a theoretical framework for sound reasoning on semantic properties of a program, establishing a correspondence between the semantics of a program, called concrete semantics, and an approximation of it, called abstract semantics. Let C and A be complete lattices, a pair of functions $\alpha : C \rightarrow A$ and $\gamma : A \rightarrow C$ forms a *Galois Connection* between C and A , written $\langle C, \sqsubseteq_C \rangle \xleftrightarrow[\alpha]{\gamma} \langle A, \sqsubseteq_A \rangle$, if $\forall c \in C, a \in A : \alpha(c) \sqsubseteq_A a \Leftrightarrow c \sqsubseteq_C \gamma(a)$. Equivalently, the Galois Connection exists if $\alpha \circ \gamma$ is reductive (i.e., if $\forall a \in A : \alpha \circ \gamma(a) \sqsubseteq_A a$), and $\gamma \circ \alpha$ is extensive (i.e., if $\forall c \in C : c \sqsubseteq_C \gamma \circ \alpha(c)$). In addition, if $\alpha \circ \gamma = id$, then α and γ form a *Galois Embedding*, written $\langle C, \sqsubseteq_C \rangle \xleftrightarrow[\alpha]{\gamma} \langle A, \sqsubseteq_A \rangle$, where no two abstract elements have the same concretization. Furthermore, if also $\gamma \circ \alpha = id$, then α and γ form a *Galois Isomorphism*, written $\langle C, \sqsubseteq_C \rangle \xleftrightarrow[\alpha]{\gamma} \langle A, \sqsubseteq_A \rangle$, where A is simply a reshaping of C and no abstraction (i.e., loss of precision) happens. Note that Abstract Interpretation can be employed also when a Galois Connection does not exist: in fact, it is sufficient that C and A are complete partial orders, and that a monotone concretization γ exists.

Soundness. Given $\langle C, \sqsubseteq_C \rangle \xleftrightarrow[\alpha]{\gamma} \langle A, \sqsubseteq_A \rangle$, a concrete function $f : C \rightarrow C$ is, in general, not computable. Hence, a function $f^\# : A \rightarrow A$ that must *correctly* approximate the function f is needed. If so, we say that the function $f^\#$ is *sound*. Given $\langle C, \sqsubseteq_C \rangle \xleftrightarrow[\alpha]{\gamma} \langle A, \sqsubseteq_A \rangle$ and a concrete function $f : C \rightarrow C$, an abstract function $f^\# : A \rightarrow A$ is sound w.r.t. f if $\forall c \in C : \alpha(f(c)) \sqsubseteq_A f^\#(\alpha(c))$, or equivalently $\forall a \in A : f(\gamma(a)) \sqsubseteq_C \gamma(f^\#(a))$. Note that the latter relation can be used to prove soundness even when a Galois Connection does not exist.

Abstract Domains. In the Abstract Interpretation framework, abstractions are defined through so-called *abstract domains*. These are composed by a partial order $\langle X, \sqsubseteq_X \rangle$, possibly extended to a complete partial order, a lattice, or a com-

plete lattice, an upper bound operator $\sqcup_X$ on X , a bottom element $\perp_X \in X$, a widening operator ∇_X that over-approximates $\sqcup_X$ and ensures the convergence on increasing chains, an *abstract transformer* $\langle \text{st} \rangle^\sharp : X \rightarrow X$ for evaluating statements, and an *abstract transformer* $\langle \text{ASM}(\mathbf{b}) \rangle^\sharp : X \rightarrow X$ for traversing conditions. The purpose of the transformers is to evolve an instance of the domain according to the semantics of the statement st to be executed, and to refine an instance of the domain assuming that a condition $\mathbf{b}$ holds.

Disjoint-Domain Unions. For functions $f : A \rightarrow Y$ and $g : B \rightarrow Z$ with $A \cap B = \emptyset$, we write $f \dot{\cup} g : A \cup B \rightarrow Y \cup Z$ for their disjoint-domain union, defined by $(f \dot{\cup} g)(x) = f(x)$ if $x \in A$, and $(f \dot{\cup} g)(x) = g(x)$ if $x \in B$.

For second-order maps $f : A \rightarrow (B \rightarrow C)$ and $g : D \rightarrow (E \rightarrow F)$, we write $f \ddot{\cup} g : A \cup D \rightarrow (B \cup E) \rightarrow (C \cup F)$ for their second-order disjoint-domain union, defined when inner maps are domain-disjoint at overlapping outer keys: $\forall x \in A \cap D : f(x) \cap g(x) = \emptyset$. For each outer key $x \in A \cup D$, $(f \ddot{\cup} g)(x) = f(x)$ if $x \in A \setminus D$, $(f \ddot{\cup} g)(x) = g(x)$ if $x \in D \setminus A$, and $(f \ddot{\cup} g)(x) = f(x) \dot{\cup} g(x)$ if $x \in A \cap D$.

Restrictions on Maps. For $f : K \rightarrow V$, $X \subseteq K$, and $Y \subseteq V$, we define the restriction on the domain as $f \upharpoonright_X = \{(x, f(x)) \mid x \in X\}$, the restriction on the co-domain as $f \upharpoonright^Y = \{(x, f(x)) \mid x \in K, f(x) \in Y\}$, and the general restriction as $f \upharpoonright_X^Y = \{(x, f(x)) \mid x \in X, f(x) \in Y\}$. For second-order maps $f : K \rightarrow (L \rightarrow V)$, with $X \subseteq K$ and $Y \subseteq V$, we define the outer-domain restriction as $f \upharpoonright_X = \{(k, f(k)) \mid k \in X\}$, the inner-co-domain restriction as $f \upharpoonright^Y = \{(k, f(k) \upharpoonright^Y) \mid k \in K\}$, and the combined double restriction as $f \upharpoonright_X^Y = \{(k, f(k) \upharpoonright^Y) \mid k \in X\}$. In both cases, the combined form equals any composition of the two: $f \upharpoonright_X^Y = f \upharpoonright_X \upharpoonright^Y = f \upharpoonright^Y \upharpoonright_X$, and $f \upharpoonright_X^Y = f \upharpoonright_X \upharpoonright^Y = f \upharpoonright^Y \upharpoonright_X$, which follow trivially from the definitions.

Let $f : K \rightarrow V$, $g : A \rightarrow B$ with $K \cap A = \emptyset$, $X_1, X_2 \subseteq K$, $Y_1, Y_2 \subseteq V$ with $Y_1 \cap Y_2 = \emptyset$, and second-order maps $M_1 : D_1 \rightarrow (L_1 \rightarrow V_1)$, $M_2 : D_2 \rightarrow (L_2 \rightarrow V_2)$ with $\forall k \in D_1 \cap D_2 : \text{dom}(M_1(k)) \cap \text{dom}(M_2(k)) = \emptyset$. In this setting, the following properties hold: (i) $f \upharpoonright_{X_1} \cup f \upharpoonright_{X_2} = f \upharpoonright_{X_1 \cup X_2}$ (*domain additivity*); (ii) $f \upharpoonright^{Y_1} \cup f \upharpoonright^{Y_2} = f \upharpoonright^{Y_1 \cup Y_2}$ (*co-domain additivity*); (iii) $(f \dot{\cup} g) \upharpoonright_X^Z = (f \upharpoonright_X^Z \dot{\cup} g \upharpoonright_X^Z)$ for any $X \subseteq K \cup A$, $Z \subseteq V \cup B$ (*distribution of $\upharpoonright$ over $\dot{\cup}$*); (iv) $(M_1 \ddot{\cup} M_2) \upharpoonright^Y = M_1 \upharpoonright^Y \ddot{\cup} M_2 \upharpoonright^Y$, for any $Y \subseteq V_1 \cup V_2$ (*distribution of $\upharpoonright$ over $\ddot{\cup}$*). Properties (i) and (iii) also hold with $\upharpoonright$ in place of $\upharpoonright$. Property (ii) holds for $\upharpoonright$ as $f \upharpoonright^{Y_1} \ddot{\cup} f \upharpoonright^{Y_2} = f \upharpoonright^{Y_1 \cup Y_2}$ (requiring $Y_1 \cap Y_2 = \emptyset$). Proofs in the version of this paper containing the technical report [9].

4 The μLL Language

The μLL language is a small, imperative language for studying memory and pointer abstractions. It extends the core of μJS [3] with a new set of features for explicit memory management and pointer manipulation, yielding a simple model that is closer to a C++-like memory model (chosen because it includes pointers, clearly defines when memory is created and accessed, and remains expressive

enough to cover a wide range of programming languages), while remaining compact for formal treatment.

The syntax of μLL is defined in Figure 1. The core expressions of μJS for arithmetic (AE), boolean (BE), string (SE), and object (OE) expressions are mostly retained, and the language is extended with memory- and pointer-specific constructs. In particular, we introduce (i) dynamic memory allocation through the **new** construct, used to allocate memory on the heap for primitive values and objects, (ii) the address-of operation $\&x$, which retrieves the memory address of a variable x , (iii) the dereference operation $\ast p$, which accesses the value stored at the memory address denoted by p , (iv) static field access $p.f$, which allows direct access to a field of an object, and (v) dynamic field access $p[s]$, which allows access to a field whose name is determined by evaluating a string expression. The last four additions form the class of *pointer expressions* (PE).

For presentation purposes, we introduce some syntactic subgroups of expressions. When objects are first defined, their fields can only contain field expressions (FE), which may include both primitive expressions and pointer expressions. Nested objects can then be added through field assignments. Heap allocation via **new** is restricted to heap-allocable expressions (HE), namely primitive expressions and objects. The non-terminal LHS denotes the set of assignable expressions. Although pointer expressions are recursive, only a syntactically restricted subset of them may occur as left-hand sides. In particular, LHS admits variables, field accesses, and dereferences, but requires the base of any field access to be an identifier, so write-mode targets are always directly addressable. Read-mode field access, by contrast, admits arbitrary pointer expressions as base, covering chains of intermediate dereferences. Since μLL has no C++-style arrow operator ($\rightarrow$), reading a field through a pointer requires an explicit dereference, as in $(\ast p).f$. Writing a field of a heap object is not directly expressible with dot notation; it requires an explicit load, field update, and write-back via dereference. This distinction will be formalized in the semantics. The non-terminal v groups expressions of primitive types, while e denotes the set of all expressions of the language, namely those that may occur on the right-hand side of a standard (non-**new**) assignment. We assume μLL programs to be *statically typed*.

As discussed in the assignment semantics (Section 4.1), the **new** operation allocates memory on the heap, while standard assignment stores values on the stack.

4.1 Concrete Semantics

In μLL , expressions evaluate to a value v in the set $\overline{\text{VAL}} \triangleq \text{VAL} \cup \text{ADDR} \cup \text{OBJ} \cup \{\perp\}$, containing values ($\text{VAL} \triangleq \mathbb{Z} \cup \{\text{true}, \text{false}\} \cup \Sigma^*$), memory addresses (ADDR), objects (OBJ), and a special value $\perp$ to denote the result of invalid computations (e.g., an access to a non-existing field of an object, the dereference of an invalid memory address, or the division by zero). $\mathbb{Z}$ denotes the set of integers, while Σ^* denotes the set of strings over the alphabet Σ . The $\perp$ value is propagated during expression evaluation: if one of the arguments of an expression is $\perp$, then the result of the whole expression is $\perp$. Identifiers are elements of the

```

 $a \in \text{AE} ::= x \mid n \mid a + a \mid a - a \mid a * a \mid a / a$ 
 $b \in \text{BE} ::= x \mid \text{true} \mid \text{false} \mid b \&\& b \mid b \mid\mid b \mid ! b \mid a < a \mid a == a \mid s == s$ 
 $s \in \text{SE} ::= x \mid "s" \mid \text{concat}(s_1, s_2)$ 
 $o \in \text{OE} ::= x \mid \{ \} \mid \{ f_0 : f_0, f_1 : f_1, \dots, f_n : f_n \}$ 
 $p \in \text{PE} ::= x \mid p[s] \mid p.f \mid \&x \mid *p$ 
 $v \in \text{VE} ::= a \mid b \mid s$ 
 $f \in \text{FE} ::= v \mid p$ 
 $h \in \text{HE} ::= v \mid o$ 
 $e \in \text{E} ::= v \mid p \mid o$ 
 $\text{lhs} \in \text{LHS} ::= x \mid x[s] \mid x.f \mid *p$ 
 $\text{st} \in \text{STMT} ::= \text{st st} \mid \text{skip}; \mid \text{lhs} = e; \mid \text{lhs} = \text{new } h; \mid \text{if } b \{ \text{st} \} \text{ else } \{ \text{st} \}$ 
 $\mid \text{while } b \{ \text{st} \}$ 

where  $x \in \text{ID}$  (identifiers),  $n \in \mathbb{Z}$ ,  $s \in \Sigma^*$ ,  $f_0, f_1, \dots, f_n \in \text{ID}$ .

```

Fig. 1. μLL syntax.

set ID and serve as both variable names and object field names; they follow the C++ naming convention (strings of letters, digits, and underscores, starting with a letter or underscore). Addresses distinguish between memory locations in the stack and in the heap, namely $\text{ADDR} \triangleq \text{ADDR}_S \cup \text{ADDR}_H$, such that $\text{ADDR}_S \cap \text{ADDR}_H = \emptyset$. ADDR_S denotes the set of stack memory addresses, where each variable $x \in \text{ID}$ is associated with a unique stack address a_x , thus $\text{ADDR}_S = \{a_x \mid x \in \text{ID}\}$. ADDR_H are memory locations in the heap, represented as h_n for each $n \in \mathbb{N}$, thus $\text{ADDR}_H = \{h_n \mid n \in \mathbb{N}\}$. The objects OBJ are structured values, represented as partial maps from field names to values, addresses, or other objects, $\text{OBJ} : \text{ID} \rightarrow \overline{\text{VAL}}$. Thus, we can represent an object literal as $\{f_0 : v_0, f_1 : v_1, \dots, f_n : v_n\}$, where for $i \in [0, n]$, $f_i \in \text{ID}$ are field names and $v_i \in \overline{\text{VAL}}$ are the values associated with the fields. The notation $\{ \}$ represents the empty object. We write $o[f \mapsto v]$ for the standard partial map override of o at field f with value v , leaving all other fields unchanged. We define the field access operator as $o(f) \triangleq v$ if $f \in \text{dom}(o)$ and $\perp$ otherwise, that retrieves the value of field f in object o .

Program memories $\chi = \langle \kappa, \eta \rangle \in \mathcal{C} \triangleq \mathcal{K} \times \mathcal{H}$ are tuples composed of a stack κ (i.e., a map from identifiers to values, memory addresses, or objects, namely $\mathcal{K} \triangleq \text{ID} \rightarrow \overline{\text{VAL}}$) and a heap η (i.e., a map from heap memory addresses to values, addresses, or objects, namely $\mathcal{H} \triangleq \text{ADDR}_H \rightarrow \overline{\text{VAL}}$). The lattice structure is given by $\langle \wp(\mathcal{C}), \subseteq \rangle$.

By static typing, each entry of κ and η consistently holds values of a single type throughout execution (a primitive value VAL, an address ADDR, or an object OBJ) and never switches between them. Abusing notation, we denote as $\perp \in \mathcal{C}$ an invalid state resulting from the assignment of a variable to $\perp$, that is propagated from one statement to the next one.

$$\begin{array}{lcl}
\frac{\forall i \in [0, n] . \llbracket f_i \rrbracket(\kappa, \eta) = v_i}{\llbracket \{f_0 : f_0, \dots, f_n : f_n\} \rrbracket(\kappa, \eta) = \{f_0 : v_0, \dots, f_n : v_n\}} & (1) & \\
\frac{}{\llbracket \{ \} \rrbracket(\kappa, \eta) = \{ \}} & (2) & \frac{\llbracket p \rrbracket(\kappa, \eta) = a \in \text{ADDR}_H}{\llbracket *p \rrbracket(\kappa, \eta) = \eta(a)} \quad (6) \\
\frac{x \in \text{dom}(\kappa)}{\llbracket x \rrbracket(\kappa, \eta) = \kappa(x)} & (3) & \frac{\llbracket p \rrbracket(\kappa, \eta) = o \in \text{OBJ}}{\llbracket p.f \rrbracket(\kappa, \eta) = o(f)} \quad (7) \\
\frac{x \in \text{dom}(\kappa)}{\llbracket \&x \rrbracket(\kappa, \eta) = a_x} & (4) & \\
\frac{\llbracket p \rrbracket(\kappa, \eta) = a_y \in \text{ADDR}_S}{\llbracket *p \rrbracket(\kappa, \eta) = \kappa(y)} & (5) & \frac{\llbracket p \rrbracket(\kappa, \eta) = o \in \text{OBJ} \quad \llbracket s \rrbracket(\kappa, \eta) = f}{\llbracket p[s] \rrbracket(\kappa, \eta) = o(f)} \quad (8) \\
\frac{\llbracket a_1 \rrbracket(\kappa, \eta) = n_1 \quad \llbracket a_2 \rrbracket(\kappa, \eta) = n_2 \quad n_2 \neq 0 \quad n_1 \bmod n_2 = 0}{\llbracket a_1 / a_2 \rrbracket(\kappa, \eta) = n_1 / n_2} & (9) &
\end{array}$$

Fig. 2. Semantics of relevant expressions in $\mathcal{C}$.

The concrete semantics of μLL statements is formalized by function $\llbracket \text{st} \rrbracket : \mathcal{C} \rightarrow \mathcal{C}$. Abusing notation, we use $\llbracket e \rrbracket : \mathcal{C} \rightarrow \overline{\text{VAL}}$ to capture the semantics of expressions, which are evaluated without side effects on the state (i.e., expression semantics does not modify the stack or heap). Note that, for all $\text{st} \in \text{STMT}$, $\llbracket \text{st} \rrbracket \perp = \perp$ and $\llbracket \text{st} \rrbracket(\kappa, \eta) = \perp$ if there exists at least one sub-expression $e \in \mathcal{E}$ in st such that $\llbracket e \rrbracket(\kappa, \eta) = \perp$. To avoid cluttering the notation, we only define $\llbracket \cdot \rrbracket$ on well-formed expressions and statements, that is, ones whose evaluation does not produce $\perp$ (i.e., where none of the arguments are $\perp$ and that produce a valid result).

Relevant Expressions. The semantic rules for relevant expressions are summarized in Figure 2. The evaluation of object expressions involves evaluating the fields of the object, associating them with their corresponding string keys. The semantics of a non-empty object (1) evaluates each expression v_i to its concrete value v_i and assigns it to their respective key f_i in the map, while the semantics of an empty object (2) evaluates to an empty map. The semantics of an identifier x (3) retrieves the value stored in the variable x in the stack. The address-of operator (4) returns the stack memory address of the variable x , which is a_x by definition. The dereference operator (5) and (6) retrieves the value stored at the memory address (both on stack or heap) given by the evaluation of p . If p does not evaluate to a valid address, the result is $\perp$. Field accesses, both static (7) and dynamic (8), retrieve the value of the field in the object, with dynamic access first evaluating the expression s used to compute the field name. If p does not evaluate to a valid object or if the field does not exist, the result is $\perp$. The arithmetic division computes the integer division of two arithmetic expressions. If the division does not yield an integer result or if the divisor is zero, the result is $\perp$.

$$\frac{\llbracket \mathbf{e} \rrbracket(\kappa, \eta) = v}{\llbracket x = \mathbf{e} \rrbracket(\kappa, \eta) = (\kappa[x \mapsto v], \eta)} \quad (10)$$

$$\frac{\llbracket \mathbf{p} \rrbracket(\kappa, \eta) = a_y \in \text{ADDR}_S \quad \llbracket \mathbf{e} \rrbracket(\kappa, \eta) = v}{\llbracket * \mathbf{p} = \mathbf{e} \rrbracket(\kappa, \eta) = (\kappa[y \mapsto v], \eta)} \quad (11)$$

$$\frac{\llbracket \mathbf{p} \rrbracket(\kappa, \eta) = a \in \text{ADDR}_H \quad \llbracket \mathbf{e} \rrbracket(\kappa, \eta) = v \in \text{VAL} \cup \text{ADDR}}{\llbracket * \mathbf{p} = \mathbf{e} \rrbracket(\kappa, \eta) = (\kappa, \eta[a \mapsto v])} \quad (12)$$

$$\frac{\kappa(x) = o \in \text{OBJ} \quad \llbracket \mathbf{e} \rrbracket(\kappa, \eta) = v}{\llbracket x.f = \mathbf{e} \rrbracket(\kappa, \eta) = (\kappa[x \mapsto o[f \mapsto v]], \eta)} \quad (13)$$

$$\frac{\kappa(x) = o \in \text{OBJ} \quad \llbracket \mathbf{e} \rrbracket(\kappa, \eta) = v \quad \llbracket \mathbf{s} \rrbracket(\kappa, \eta) = f \in \text{ID}}{\llbracket x[\mathbf{s}] = \mathbf{e} \rrbracket(\kappa, \eta) = (\kappa[x \mapsto o[f \mapsto v]], \eta)} \quad (14)$$

Fig. 3. Semantics of expression assignments $\text{lhs} = \mathbf{e}$ in $\mathcal{C}$.

Assignments. Since most statements are standard, we omit their semantics from this paper, focusing only on assignments. Figures 3 and 4 summarize the semantics of assignment statements for two different cases: the stack assignment $\text{lhs} = \mathbf{e}$, and the heap assignment $\text{lhs} = \mathbf{new} \ h$, with the latter dynamically allocating memory (on the heap) for a variable using a fresh address. This address is obtained from the function $\text{fresh}_C^H : \mathcal{H} \rightarrow \text{ADDR}_H$, which returns a new heap address $a_f \in \text{ADDR}_H$ that is not already in use in the heap.

Each figure outlines the possible left-hand side targets of the assignment: to a variable (10) and (15), to a pointer dereference (11), (12), (16) and (17), to a static field of an object (13) and (18), and to a dynamic field of an object (14) and (19). In general, every assignment operation evaluates the expression on the right-hand side, which results in a value $v \in \overline{\text{VAL}}$. In the case of heap allocations, a fresh address $a_f \in \text{ADDR}_H$ is computed, and the value produced by the evaluation of the right-hand side is stored as image of that address in η . In the following, we refer to the result of the right-hand side evaluation in stack assignments and to the freshly generated address in heap assignments as the *assigned value*, to unify the presentation of the semantics of the two types of assignments. Variable assignments store the assigned value as the variable's image in the stack. Dereference assignments store the assigned value at the variable (on the stack) or address (on the heap) obtained by evaluating the left-hand side dereference. Field assignments require the base identifier to hold an object directly on the stack: the object is retrieved, updated to map the desired field to the assigned value, and stored back. To assign a field of a heap-allocated object (or a stack-allocated object pointed to by a pointer), an explicit load/store sequence is required: load the object via dereference, update the field, and write it back. Note that objects stored at heap addresses are always flat, i.e., their field values lie in $\text{VAL} \cup \text{ADDR}$: this is enforced jointly by rule (12), which restricts the right-hand side of a heap dereference assignment to $\text{VAL} \cup \text{ADDR}$, and by the grammar, which limits the fields of object literals to FE (primitive values and pointer expressions).

$$\frac{a_f = \text{fresh}_c^H(\eta) \quad \llbracket \mathbf{h} \rrbracket(\kappa, \eta) = v}{\llbracket x = \text{new } \mathbf{h} \rrbracket(\kappa, \eta) = (\kappa[x \mapsto a_f], \eta[a_f \mapsto v])} \quad (15)$$

$$\frac{\llbracket \mathbf{p} \rrbracket(\kappa, \eta) = a_y \in \text{ADDRS} \quad a_f = \text{fresh}_c^H(\eta) \quad \llbracket \mathbf{h} \rrbracket(\kappa, \eta) = v}{\llbracket * \mathbf{p} = \text{new } \mathbf{h} \rrbracket(\kappa, \eta) = (\kappa[y \mapsto a_f], \eta[a_f \mapsto v])} \quad (16)$$

$$\frac{\llbracket \mathbf{p} \rrbracket(\kappa, \eta) = a \in \text{ADDRH} \quad a_f = \text{fresh}_c^H(\eta) \quad \llbracket \mathbf{h} \rrbracket(\kappa, \eta) = v}{\llbracket * \mathbf{p} = \text{new } \mathbf{h} \rrbracket(\kappa, \eta) = (\kappa, \eta[a \mapsto a_f, a_f \mapsto v])} \quad (17)$$

$$\frac{a_f = \text{fresh}_c^H(\eta) \quad \llbracket \mathbf{h} \rrbracket(\kappa, \eta) = v \quad \kappa(x) = o \in \text{OBJ}}{\llbracket x.f = \text{new } \mathbf{h} \rrbracket(\kappa, \eta) = (\kappa[x \mapsto o[f \mapsto a_f]], \eta[a_f \mapsto v])} \quad (18)$$

$$\frac{a_f = \text{fresh}_c^H(\eta) \quad \llbracket \mathbf{h} \rrbracket(\kappa, \eta) = v \quad \kappa(x) = o \in \text{OBJ} \quad \llbracket \mathbf{s} \rrbracket(\kappa, \eta) = f}{\llbracket x[\mathbf{s}] = \text{new } \mathbf{h} \rrbracket(\kappa, \eta) = (\kappa[x \mapsto o[f \mapsto a_f]], \eta[a_f \mapsto v])} \quad (19)$$

Fig. 4. Semantics of `new` assignments `lhs = new h` in $\mathcal{C}$

Assume Statement. The `ASM(b)` statement is an auxiliary construct, not part of the μLL syntax, used when analyzing conditionals and loops to restrict the program state to traces satisfying a guard. Its concrete semantics is:

$$(\text{assume}) \quad \frac{\llbracket \mathbf{b} \rrbracket(\kappa, \eta) = \text{true}}{\llbracket \text{ASM}(\mathbf{b}) \rrbracket(\kappa, \eta) = (\kappa, \eta)}$$

States for which the guard evaluates to `false` are mapped to $\perp$.

Example 1 (μLL Running Example). We now encode the motivating C snippet from Listing 1 into μLL . The translation follows a few systematic choices. First, the C struct initialisation `list.L=0; list.R=0;` becomes simply `list = {}`: since μLL is statically typed and L and R are address-typed fields, they cannot hold the integer 0 as a null placeholder without violating the typing invariant. Instead, a null-initialised pointer field is represented as absent from the object. Second, pointer declarations (`struct node *x = &list`) map directly to `x = &list`, using the address-of operator already in the language. Third, the `malloc` call combined with field initialisation (`n->L=tail; n->R=0`) is folded into a single heap-allocation statement `n = new { L:tail }`,² which atomically allocates a fresh heap address and stores the object; the R field is omitted for the same reason as above. Fourth, mutation through a pointer – the C idiom `tail->R = n` – requires an explicit load/store sequence: `tmp = *tail` loads the object, `tmp.R = n` updates the field, and `*tail = tmp` writes it back. Finally, the two non-deterministic booleans `__VERIFIER_nondet_bool()` are replaced by the predicates `nd_loop()` and `nd_pick()` that yield non-deterministic values.³

² The μLL language does not support uninitialized data structures, so we perform explicit initialisation of all fields at allocation time. However, this is not a limitation of the framework, but rather a stylistic choice to simplify the presentation.

³ While μLL does not support predicates, we rely on an intuitive definition to faithfully replicate the running example. As these do not represent operations of interest for our framework, we omit their definition.

Table 1. Concrete state annotations for Listing 2. κ_l and η_l denote the stack and heap at line l . Inside the loop body, $a_f^i = \text{fresh}_c^H(\eta_6^i)$ and $i \geq 1$.

Line	κ	η
1	$[\text{list} \mapsto \{ \}]$	$\emptyset$
2	$\kappa_1[\mathbf{x} \mapsto a_{\text{list}}]$	
3	$\kappa_2[\text{tail} \mapsto a_{\text{list}}]$	
6	κ_{17}^{i-1}	η_{17}^{i-1}
7	$\kappa_6^i[\mathbf{n} \mapsto a_f^i]$	$\eta_6^i[a_f^i \mapsto \{L: a_{\text{list}}\}]$
8	$\kappa_7^i[\text{tmp} \mapsto \kappa_7^i(\text{list})]$	
9	$\kappa_8^i[\text{tmp} \mapsto \kappa_8^i(\text{tmp})[\mathbf{R} \mapsto a_f^i]]$	
10	$\kappa_9^i[\text{list} \mapsto \kappa_9^i(\text{tmp})]$	
13	$\kappa_{10}^i[\mathbf{x} \mapsto a_f^i]$	
15	κ_{10}^i	
17	$\kappa_{13}^i \cup \kappa_{15}^i$	

Listing 2. μLL translation of Listing 1 with concrete state annotations.

```

1 list = {};
2 x = &list;
3 tail = &list;
4
5 /* nd_loop() and nd_pick() denote nondeterministic booleans */
6 while nd_loop() {
7     n = new { L:tail };
8     tmp = *tail;
9     tmp.R = n;
10    *tail = tmp;
11
12    if nd_pick() {
13        x = n;
14    } else {
15        skip;
16    }
17 }
```

The state annotations in Table 1 track the concrete state at each program point. Before the loop (lines 1–3), the stack maps `list` to an empty object and both `x` and `tail` to a_{list} , with the heap empty. At each iteration i , a fresh heap address a_f^i is allocated for the new node (line 7), `list.R` is updated to a_f^i via the load/store sequence on lines 8–10, and `x` either advances to a_f^i (line 13) or retains its previous value (line 15), so after the loop `x` may hold a_{list} or any of $a_f^1, \dots, a_f^k$.

4.2 Collecting Semantics

The concrete semantics is lifted pointwise to sets of states $X \subseteq \mathcal{C}$. For a statement st , $\llbracket \text{st} \rrbracket^\wp : \wp(\mathcal{C}) \rightarrow \wp(\mathcal{C})$ is defined as $\llbracket \text{st} \rrbracket^\wp(X) = \{\llbracket \text{st} \rrbracket(\chi) \mid \chi \in X\}$; for the auxiliary construct $\text{ASM}(\mathbf{b})$, $\llbracket \text{ASM}(\mathbf{b}) \rrbracket^\wp : \wp(\mathcal{C}) \rightarrow \wp(\mathcal{C})$ is defined as $\llbracket \text{ASM}(\mathbf{b}) \rrbracket^\wp(X) = \{\chi' \mid \chi \in X \wedge \llbracket \text{ASM}(\mathbf{b}) \rrbracket(\chi) = \chi' \wedge \chi' \neq \perp\}$; and for an expression $\mathbf{e}$, $\llbracket \mathbf{e} \rrbracket^\wp : \wp(\mathcal{C}) \rightarrow \wp(\text{VAL})$ is defined as $\llbracket \mathbf{e} \rrbracket^\wp(X) = \{\llbracket \mathbf{e} \rrbracket(\chi) \mid \chi \in X\}$.

5 Split State

A state $\sigma \in \mathcal{S}$ is a tuple $(\nu, \mu) \in \mathcal{V} \times \mathcal{M}$ where locations (ID or ADDR_H) are partitioned by the kind of information they contain: (i) if the location contains a concrete value, it is stored in $\mathcal{V}$, (ii) otherwise, if it contains an address, it is stored in $\mathcal{M}$. Moreover, if a location contains an object, the latter is *exploded* into entries identified by the location and the field name, each stored in $\mathcal{V}$ or $\mathcal{M}$ using the same reasoning (each object is flattened to per-field mappings via our `explode` function, see Section 5.1). Here, $\text{ADDR}_\text{R} = \{r_n \mid n \in \mathbb{N}\}$ is a set of addresses disjoint from all concrete addresses and identifiers ($\text{ADDR}_\text{R} \cap \text{ADDR} = \emptyset$, $\text{ADDR}_\text{R} \cap \text{ID} = \emptyset$), generated during object decomposition to represent the locations of subobjects within stack-allocated objects. We define $\text{ADDR}^+ \triangleq \text{ADDR} \cup \text{ADDR}_\text{R}$ as the extended address set of the split state. The $\mathcal{S}$ subcomponents are formalized as:

$$\begin{aligned} \mathcal{V} &= \mathcal{V}_{ih} \times \mathcal{V}_f \\ \mathcal{V}_{ih} &: (\text{ID} \cup \text{ADDR}_\text{H}) \rightarrow \text{VAL} \\ \mathcal{V}_f &: (\text{ID} \cup \text{ADDR}_\text{H} \cup \text{ADDR}_\text{R}) \rightarrow (\text{ID} \rightarrow \text{VAL}) \end{aligned}$$

$$\begin{aligned} \mathcal{M} &= \mathcal{M}_{ih} \times \mathcal{M}_f \\ \mathcal{M}_{ih} &: (\text{ID} \cup \text{ADDR}_\text{H}) \rightarrow \text{ADDR} \\ \mathcal{M}_f &: (\text{ID} \cup \text{ADDR}_\text{H} \cup \text{ADDR}_\text{R}) \rightarrow (\text{ID} \rightarrow \text{ADDR}^+) \end{aligned}$$

where the subscript *ih* represents storage for variables and heap locations, and *f* for object fields. A split state can be equivalently referred to as σ , as the pair (ν, μ) , or as the expanded form $((\nu_{ih}, \nu_f), (\mu_{ih}, \mu_f))$. The lattice structure is given by $\langle \wp(\mathcal{S}), \subseteq \rangle$.

Throughout, all states are assumed to satisfy the following well-formedness condition.

Definition 1 (Well-Formed Split State). *Consider an arbitrary split state $\sigma = ((\nu_{ih}, \nu_f), (\mu_{ih}, \mu_f))$. We say that σ is well-formed iff:*

- the pairs of functions $(\nu_{ih}, \mu_{ih}), (\nu_{ih}, \nu_f), (\nu_{ih}, \mu_f), (\mu_{ih}, \nu_f), (\mu_{ih}, \mu_f)$ have disjoint domains, and
- $\forall l \in \text{dom}(\nu_f) \cup \text{dom}(\mu_f)$, the pair of functions $(\nu_f(l), \mu_f(l))$ have disjoint domains.

Note that this is a reasonable assumption, since in a concrete state, a location can only be mapped to a value, an address, or an object. Thus, when partitioning this state, each location can only end up in one of the split components. Since $\vdash$ retains all keys (as used in α_S , Section 5.1), every object-holding location l appears in both $\text{dom}(\nu_f)$ and $\text{dom}(\mu_f)$, while each *object-field pair* (l, f) occurs in exactly one of the two inner maps. Concretely:

- *Empty object* $\{ \}$: $\nu_f(l) = \emptyset$ and $\mu_f(l) = \emptyset$.
- *Only value-typed fields* (e.g., $\{f:v\}$, $v \in \text{VAL}$): $\nu_f(l) \neq \emptyset$ and $\mu_f(l) = \emptyset$.
- *Only reference-typed fields* (e.g., $\{f:a\}$, $a \in \text{ADDR}^+$): $\nu_f(l) = \emptyset$ and $\mu_f(l) \neq \emptyset$.
- *Mixed fields*: both $\nu_f(l) \neq \emptyset$ and $\mu_f(l) \neq \emptyset$.

Example 2 (Split State Benefit). Consider the program $p = \text{new } 5; \text{tmp} = *p; y = \text{tmp} + 1$. After all three statements, the concrete state is the pair (κ, η) with $\kappa = \{p \mapsto a, \text{tmp} \mapsto 5, y \mapsto 6\}$ and $\eta = \{a \mapsto 5\}$; within κ , an address and two integer values coexist in the same map. The split state separates these by type into two components: $\mu_{ih} = \{p \mapsto a\}$ captures the memory structure, while $\nu_{ih} = \{a \mapsto 5, \text{tmp} \mapsto 5, y \mapsto 6\}$ captures the numerical values; the fields components μ_f and ν_f are empty here. This separation is not merely convenient but necessary from a formalization standpoint: each abstraction step can then focus exclusively on the relevant component of the state, i.e., memory structure μ or numerical values ν , without needing to inspect the other.

5.1 Concrete-Split Equivalence

To ensure that the split state and semantics are just a reformulation of the concrete state and semantics, it is necessary to show that there is no loss of precision in the abstraction/concretization process, nor in the split semantics. This reduces to showing: (i) there exist a pair of monotone functions α_S and γ_S such that $\langle \wp(\mathcal{C}), \subseteq \rangle \xleftrightarrow[\gamma_S]{\alpha_S} \langle \wp(\mathcal{S}), \subseteq \rangle$, and (ii) for all statements st and split states (ν, μ) , $\llbracket \text{st} \rrbracket(\gamma_S(\nu, \mu)) = \gamma_S(\llbracket \text{st} \rrbracket(\nu, \mu))$.

Exploding and Imploding Objects. In order to define the relationship between concrete and split states, we introduce two mutually inverse helper functions **explode** : $((\text{ID} \cup \text{ADDR}_H) \rightarrow \text{OBJ}) \rightarrow ((\text{ID} \cup \text{ADDR}_H \cup \text{ADDR}_R) \rightarrow \text{OBJ})$ and **implode** : $((\text{ID} \cup \text{ADDR}_H \cup \text{ADDR}_R) \rightarrow \text{OBJ}) \rightarrow ((\text{ID} \cup \text{ADDR}_H) \rightarrow \text{OBJ})$, where $\text{OBJ} \triangleq \text{ID} \rightarrow (\text{VAL} \cup \text{ADDR}^+ \cup \perp)$ denotes an *exploded object* – a flat field map with no nested objects. Intuitively, **explode** flattens nested objects replacing each sub-object with a fresh address $r \in \text{ADDR}_R$, while **implode** reassembles them back. More precisely, **explode** introduces ADDR_R addresses only for *stack* objects (indexed by ID), and acts as the identity on *heap* objects (indexed by ADDR_H): the latter are already flat, since the **new** construct allocates each nested object at a fresh heap address in ADDR_H (see Section 4.1). Formal definitions and proofs are in the version of this paper containing the technical report [9].

Concretization and Abstraction. In the following definitions we use disjoint-domain union $f \dot{\cup} g$, second-order disjoint-domain union $f \ddot{\cup} g$, restriction $f \upharpoonright_X$, $f \upharpoonright^Y$, $f \upharpoonright_X^Y$, and double restriction $f \upharpoonright_X^Y$, as introduced in Section 3.

Definition 2 (Concretization of Split States). *The function $\gamma_S : \wp(\mathcal{S}) \rightarrow \wp(\mathcal{C})$ maps a set of split states Σ to a set of concrete states. It is defined as:*

$$\gamma_S(\Sigma) = \bigcup_{\sigma \in \Sigma} \{\gamma_S(\sigma)\},$$

where $\gamma_S : \mathcal{S} \rightarrow \mathcal{C}$ is the concretization function for a single split state. Given a split state $\sigma = (\nu, \mu)$, the corresponding concrete state is:

$$\gamma_S(\sigma) = (\nu_i \dot{\cup} \mu_i \dot{\cup} \text{implode}(\nu_{sf} \ddot{\cup} \mu_{sf}), \nu_h \dot{\cup} \mu_h \dot{\cup} \text{implode}(\nu_{hf} \ddot{\cup} \mu_{hf})).$$

where

$$\nu_i = \nu_{ih} \upharpoonright_{\text{ID}}, \quad \nu_h = \nu_{ih} \upharpoonright_{\text{ADDR}_H}, \quad \nu_{sf} = \nu_f \upharpoonright_{(\text{ID} \cup \text{ADDR}_R)}, \quad \nu_{hf} = \nu_f \upharpoonright_{\text{ADDR}_H},$$

$$\mu_i = \mu_{ih} \upharpoonright_{\text{ID}}, \quad \mu_h = \mu_{ih} \upharpoonright_{\text{ADDR}_H}, \quad \mu_{sf} = \mu_f \upharpoonright_{(\text{ID} \cup \text{ADDR}_R)}, \quad \mu_{hf} = \mu_f \upharpoonright_{\text{ADDR}_H}.$$

Definition 3 (Abstraction of Concrete States). *The function $\alpha_S : \wp(\mathcal{C}) \rightarrow \wp(\mathcal{S})$ maps a set of concrete states X to a set of split states. It is defined as:*

$$\alpha_S(X) = \bigcup_{\chi \in X} \{\dot{\alpha}_S(\chi)\},$$

where $\dot{\alpha}_S : \mathcal{C} \rightarrow \mathcal{S}$ is the abstraction function for a single concrete state. Given a concrete state $\chi = (\kappa, \eta)$, the corresponding split state is:

$$\dot{\alpha}_S(\chi) = ((\nu_{ih}, \nu_f), (\mu_{ih}, \mu_f)),$$

where, letting $\tilde{O} \triangleq \text{explode}(\kappa \upharpoonright^{\text{OBJ}} \dot{\cup} \eta \upharpoonright^{\text{OBJ}})$:

$$\nu_{ih} = \kappa \upharpoonright^{\text{VAL}} \dot{\cup} \eta \upharpoonright^{\text{VAL}}, \quad \nu_f = \tilde{O} \upharpoonright^{\text{VAL}},$$

$$\mu_{ih} = \kappa \upharpoonright^{\text{ADDR}} \dot{\cup} \eta \upharpoonright^{\text{ADDR}}, \quad \mu_f = \tilde{O} \upharpoonright^{\text{ADDR}^+}.$$

Theorem 1 (Concrete-Split Galois Isomorphism). *The functions α_S and γ_S form a Galois isomorphism between the concrete state lattice and the split state lattice: $\langle \wp(\mathcal{C}), \subseteq \rangle \xleftrightarrow[\alpha_S]{\gamma_S} \langle \wp(\mathcal{S}), \subseteq \rangle$. The proof is provided in the technical report [9].*

Split Semantics. Rather than defining new semantics directly on the split state, we follow [35] and leverage the fact that α_S and γ_S form a Galois isomorphism between concrete and split states (Theorem 1). This property ensures that applying γ_S to a split state, computing semantics in the concrete domain, and then lifting the result back via α_S yields the most precise abstraction of the

concrete outcome [14]. Note that this does not make our semantics computable, but allows direct reuse of the concrete semantics as a specification for the split semantics without creating a new one. Successive abstractions can thus compare their abstract semantics to the concrete one by applying α_S and γ_S . Formally, the split semantics of a statement st , auxiliary construct $\text{ASM}(\mathbf{b})$, and expression $\mathbf{e}$ are respectively: $\llbracket \text{st} \rrbracket = \alpha_S \circ \llbracket \text{st} \rrbracket \circ \gamma_S$, $\llbracket \text{ASM}(\mathbf{b}) \rrbracket = \alpha_S \circ \llbracket \text{ASM}(\mathbf{b}) \rrbracket \circ \gamma_S$, and $\llbracket \mathbf{e} \rrbracket = \llbracket \mathbf{e} \rrbracket \circ \gamma_S$. Note that for expressions, α_S is unnecessary since $\llbracket \mathbf{e} \rrbracket$ already produces a concrete value. Since γ_S and α_S are inverses of each other (as shown in the technical report [9]), the following equivalence holds trivially.

Theorem 2 (Split and Concrete Semantics Equivalence). *For any split state $\sigma \in \mathcal{S}$, the following holds: (i) for any $\text{st} \in \text{STMT}$: $\llbracket \text{st} \rrbracket \gamma_S(\sigma) = \gamma_S(\llbracket \text{st} \rrbracket(\sigma))$; (ii) for any $\mathbf{b} \in \text{BE}$: $\llbracket \text{ASM}(\mathbf{b}) \rrbracket \gamma_S(\sigma) = \gamma_S(\llbracket \text{ASM}(\mathbf{b}) \rrbracket(\sigma))$; (iii) for any $\mathbf{e} \in \text{E}$: $\llbracket \mathbf{e} \rrbracket \gamma_S(\sigma) = \llbracket \mathbf{e} \rrbracket(\sigma)$. (Proof in the technical report [9]).*

Thus, the split state is a rephrasing of the concrete state, with no loss of precision in both the commutation between the representations and the semantics. Abstractions over the concrete semantics are challenging: the program state is composed by two functions that mix value information with points-to information. Instead, the split state provides a setting in which value and points-to information are well separated: this lays the ground for modular abstractions of the split state, in which each component can focus on one kind of information only, either points-to or values. Since the split state is a simple rephrasing, being sound w.r.t. the split semantics entails being sound w.r.t. the concrete semantics.

5.2 Collecting Semantics

The split semantics is lifted pointwise to sets of split states $\Sigma \subseteq \mathcal{S}$. For a statement st , $\llbracket \text{st} \rrbracket^\varphi : \wp(\mathcal{S}) \rightarrow \wp(\mathcal{S})$ is defined as $\llbracket \text{st} \rrbracket^\varphi(\Sigma) = \{\llbracket \text{st} \rrbracket(\sigma) \mid \sigma \in \Sigma\}$; for the auxiliary construct $\text{ASM}(\mathbf{b})$, $\llbracket \text{ASM}(\mathbf{b}) \rrbracket^\varphi : \wp(\mathcal{S}) \rightarrow \wp(\mathcal{S})$ is defined as $\llbracket \text{ASM}(\mathbf{b}) \rrbracket^\varphi(\Sigma) = \{\sigma' \mid \sigma \in \Sigma \wedge \llbracket \text{ASM}(\mathbf{b}) \rrbracket(\sigma) = \sigma' \wedge \sigma' \neq \perp\}$; and for an expression $\mathbf{e}$, $\llbracket \mathbf{e} \rrbracket^\varphi : \wp(\mathcal{S}) \rightarrow \wp(\text{VAL})$ is defined as $\llbracket \mathbf{e} \rrbracket^\varphi(\Sigma) = \{\llbracket \mathbf{e} \rrbracket(\sigma) \mid \sigma \in \Sigma\}$. By pointwise application of Theorem 2 over all $\sigma \in \Sigma$, the split collecting semantics commutes with concretization: $\gamma_S(\llbracket \text{st} \rrbracket^\varphi(\Sigma)) = \llbracket \text{st} \rrbracket^\varphi(\gamma_S(\Sigma))$.

6 Abstract State

In this section, we define a general framework in which two abstract domains, one abstracting values and one abstracting memory, can cooperate modularly to form a sound over-approximation of the split state and semantics. Specifically, we consider a *value domain* $\mathcal{V}^\sharp$, which abstracts the value component ν of the split state $(\nu, \mu) \in \mathcal{V} \times \mathcal{M}$, and a *memory domain* $\mathcal{M}^\sharp$, which abstracts the memory component μ . Intuitively, the value domain approximates the values stored in program variables and memory locations, while the memory domain approximates the structure of memory, including allocation, indirection, aliasing, and object fields. Although $\mathcal{M}^\sharp$ is often referred to in the literature as a *heap*

domain/state, in our framework it abstracts all program memory locations, both stack- and heap-allocated. For this reason, we consistently use the terms *memory domain/state* to refer to $\mathcal{M}^\sharp$.

Memory Identifiers and Domain Interaction. The cooperation between the two domains is mediated through a set of *memory identifiers*, denoted by `MemID`, along with a set of operations defined by the domains to manipulate them. These identifiers represent abstract memory locations corresponding to one or more split locations in the memory of the analyzed program, including both stack-allocated and heap-allocated entities. They are treated as abstract names without internal structure. Their concrete interpretation and allocation strategy are determined by the chosen memory domain implementation.

During abstract execution, memory operations may reorganize memory identifiers, for instance by materializing new locations or merging existing ones. Such structural changes are communicated to the value domain through *substitutions*, ensuring that the combined abstract state remains consistent and sound.

Domain Requirements. Since the framework is parametric in the choice of $\mathcal{V}^\sharp$ and $\mathcal{M}^\sharp$, we require each abstract domain $\mathcal{X} \in \{\mathcal{V}^\sharp, \mathcal{M}^\sharp\}$ to satisfy the following minimal properties, using the abstract interpretation terminology recalled in Section 3:

- it should form a partial order, denoted $\langle \mathcal{X}, \sqsubseteq_{\mathcal{X}} \rangle$, with bottom element $\perp_{\mathcal{X}}$, and should define an upper bound operator $\sqcup_{\mathcal{X}}$;
- it should provide a widening operator $\nabla_{\mathcal{X}}$ (if not needed by the domain itself, it can simply delegate to $\sqcup_{\mathcal{X}}$).

In addition, we require the domains to provide monotone concretization functions $\gamma_{\mathcal{V}^\sharp}$ and $\gamma_{\mathcal{M}^\sharp}$ whose purpose is not to directly abstract parts of the split state, but to provide partial views of the program memory that can be combined to produce an overall concretization of the abstract state. Furthermore, instead of defining standard abstract transformers, we require each domain to expose a small operational interface that the framework can use to evolve the two domains consistently during the analysis. These requirements are detailed in Sections 6.2 and 6.3, respectively.

6.1 Abstract State Construction

The overall abstract state is obtained by combining the value and memory domains into a single state. It is defined as $\mathcal{S}^\sharp \triangleq \mathcal{V}^\sharp \otimes \mathcal{M}^\sharp$, where $\otimes$ denotes the *smash product* of abstract domains [2], a form of reduced product [13] that propagates the bottom element of one component to all others. An abstract state is therefore a pair $\sigma^\sharp = (\nu^\sharp, \mu^\sharp) \in \mathcal{S}^\sharp$, where $\nu^\sharp \in \mathcal{V}^\sharp$ and $\mu^\sharp \in \mathcal{M}^\sharp$. Since the smash product is a special form of Cartesian product, the ordering and lattice operators on $\mathcal{S}^\sharp$ are defined component-wise:

$$-(\nu_1^\sharp, \mu_1^\sharp) \sqsubseteq_{\mathcal{S}^\sharp} (\nu_2^\sharp, \mu_2^\sharp) \iff \nu_1^\sharp \sqsubseteq_{\mathcal{V}^\sharp} \nu_2^\sharp \wedge \mu_1^\sharp \sqsubseteq_{\mathcal{M}^\sharp} \mu_2^\sharp;$$

$$\begin{aligned}
& - (\nu_1^\sharp, \mu_1^\sharp) \sqcup_{\mathcal{S}^\sharp} (\nu_2^\sharp, \mu_2^\sharp) = (\nu_1^\sharp \sqcup_{\mathcal{V}^\sharp} \nu_2^\sharp, \mu_1^\sharp \sqcup_{\mathcal{M}^\sharp} \mu_2^\sharp); \\
& - (\nu_1^\sharp, \mu_1^\sharp) \nabla_{\mathcal{S}^\sharp} (\nu_2^\sharp, \mu_2^\sharp) = (\nu_1^\sharp \nabla_{\mathcal{V}^\sharp} \nu_2^\sharp, \mu_1^\sharp \nabla_{\mathcal{M}^\sharp} \mu_2^\sharp).
\end{aligned}$$

For the same reason, $\mathcal{S}^\sharp$ inherits the lattice properties of its components: $\sqsubseteq_{\mathcal{S}^\sharp}$ is a partial order, $\sqcup_{\mathcal{S}^\sharp}$ is an upper bound operator, $\perp_{\mathcal{S}^\sharp}$ is the bottom element, and $\nabla_{\mathcal{S}^\sharp}$ is a widening operator.

6.2 Concretization Function

The concretization function $\gamma_{\mathcal{S}^\sharp} : \mathcal{S}^\sharp \rightarrow \wp(\mathcal{S})$ maps an abstract state to the set of all possible split states it represents.

$\gamma_{\mathcal{V}^\sharp}$ is a monotone function that produces split value environments in which split addresses are abstracted by memory identifiers. Concretely, we write:

$$\gamma_{\mathcal{V}^\sharp} : \mathcal{V}^\sharp \rightarrow \wp(\mathcal{V}^{\text{ID}} \times \mathcal{V}^{\text{MemID}}),$$

where $\mathcal{V}^{\text{MemID}} : \text{MemID} \rightarrow \text{VAL}$ is a mapping that associates split values to memory identifiers, while $\mathcal{V}^{\text{ID}}$ denotes the ID-indexed part of the value split state, i.e. the restriction of both $\mathcal{V}_{ih}$ and $\mathcal{V}_f$ to keys in ID (so $\text{dom}(\mathcal{V}^{\text{ID}}) \subseteq \text{ID}$). To relate abstract memory identifiers to split memory locations, the concretization of the memory domain provides, together with split memory structure, a mapping that connects abstract identifiers to split addresses. Formally, we define:

$$\gamma_{\mathcal{M}^\sharp} : \mathcal{M}^\sharp \rightarrow \wp(\mathcal{M} \times \gamma_{\text{MemID}} \times \gamma_{\text{MemID}_f}),$$

as a monotone function. Intuitively, for each abstract memory state $\mu^\sharp$, every element $(\mu, \gamma_{\text{MemID}}, \gamma_{\text{MemID}_f}) \in \gamma_{\mathcal{M}^\sharp}(\mu^\sharp)$ describes: (i) a possible split memory structure μ , and (ii) how abstract memory identifiers are implemented by split memory locations in μ . Here, $\gamma_{\text{MemID}} : \text{MemID} \rightarrow \wp(\text{ADDR}_H)$ maps abstract memory identifiers to sets of heap addresses, while $\gamma_{\text{MemID}_f} : \text{MemID} \rightarrow \wp((\text{ADDR}_H \cup \text{ADDR}_R) \times \text{ID})$ maps abstract memory identifiers to sets of split field locations (only heap- and stack-nested objects carry fields). γ_{MemID} and γ_{MemID_f} have disjoint domains, namely $\text{dom}(\gamma_{\text{MemID}}) \cap \text{dom}(\gamma_{\text{MemID}_f}) = \emptyset$: by the static typing assumption (Section 4), the abstract memory identifiers for base locations (domain of γ_{MemID}) and for field locations (domain of γ_{MemID_f}) are always distinct.

To concretize the MemID-indexed value component $\mathcal{V}^{\text{MemID}}$, we need to replace each memory identifier in the state with the split locations it may denote. We thus define two helper concretizations that replace memory identifiers in $\nu \in \mathcal{V}^{\text{MemID}}$ by split bindings induced by γ_{MemID} and γ_{MemID_f} , defined as:

$$\begin{aligned}
\rho_b & : (\mathcal{V}^{\text{MemID}} \times \gamma_{\text{MemID}}) \rightarrow (\text{ADDR}_H \rightarrow \text{VAL}) \\
\rho_b(\nu, \gamma_{\text{MemID}}) & = \{(a, v) : (\iota, v) \in \nu \wedge a \in \gamma_{\text{MemID}}(\iota)\}
\end{aligned}$$

$$\begin{aligned}
\rho_f & : (\mathcal{V}^{\text{MemID}} \times \gamma_{\text{MemID}_f}) \rightarrow ((\text{ADDR}_H \cup \text{ADDR}_R) \rightarrow \text{ID} \rightarrow \text{VAL}) \\
\rho_f(\nu, \gamma_{\text{MemID}_f}) & = \{(a, \{(f_i, v_i)\}) : (\iota, v_i) \in \nu \wedge (a, f_i) \in \gamma_{\text{MemID}_f}(\iota)\}
\end{aligned}$$

Both functions are trivially monotone, since their definitions only add bindings to the output as the inputs grow.

Finally, the concretization of the full abstract state $(\nu^\#, \mu^\#)$ is defined as

$$\begin{aligned} \gamma_{S^\#}(\nu^\#, \mu^\#) = & \{(\nu, \mu) : (\nu_{id}, \nu_{mid}) \in \gamma_{V^\#}(\nu^\#) \wedge (\mu, \gamma_{MemID}, \gamma_{MemIDf}) \in \gamma_{M^\#}(\mu^\#) \\ & \wedge \nu_{mid}^{id} = \rho_b(\nu_{mid}, \gamma_{MemID}) \wedge \nu_{mid}^{idf} = \rho_f(\nu_{mid}, \gamma_{MemIDf}) \\ & \wedge \nu = \nu_{id} \dot{\cup} \nu_{mid}^{id} \dot{\cup} \nu_{mid}^{idf}\} \end{aligned}$$

Here, $(\nu_{id}, \nu_{mid}) \in \gamma_{V^\#}(\nu^\#)$ is a pair produced by the value concretization, where $\nu_{id} \in \mathcal{V}^{ID}$ provides the ID-keyed bindings directly, and $\nu_{mid} \in \mathcal{V}^{MemID}$ provides the MemID-keyed bindings that still need to be resolved to split addresses. The helpers ρ_b and ρ_f perform that resolution: ν_{mid}^{id} maps memory identifiers to base-address bindings, while ν_{mid}^{idf} maps them to location-field bindings. The final split value environment ν is the disjoint union of all three components.

Lemma 1 (Monotonicity of $\gamma_{S^\#}$). *The concretization function $\gamma_{S^\#}$ is monotone, that is:*

$$\forall \sigma_1^\#, \sigma_2^\# \in \mathcal{S}^\# : \sigma_1^\# \sqsubseteq_{\mathcal{S}^\#} \sigma_2^\# \implies \gamma_{S^\#}(\sigma_1^\#) \subseteq \gamma_{S^\#}(\sigma_2^\#)$$

Intuitively, since $\mathcal{S}^\#$ is a product domain, monotonicity follows if $\gamma_{V^\#}$ and $\gamma_{M^\#}$ are monotone. Proof in the version of this paper containing the technical report [9].

6.3 Abstract Semantics

To formalize the semantics of our framework, we require that the value and memory domains provide specialized abstract transformers that the framework exploits to provide a sound over-approximation of the split semantics. These are either slight modifications of the standard transformers required by abstract interpretation, or new operators that extract information from the domains. To simplify the presentation of such transformers we introduce the following auxiliary sets: (i) $\tilde{V} \triangleq \text{MemID} \cup \text{VE}$, the set of value expressions that can also contain memory identifiers; (ii) $\text{RHS} \triangleq \text{e} \mid \text{o} \mid \text{new}(\text{a} \mid \text{b} \mid \text{s} \mid \text{o})$, the set of expressions that may occur on the right-hand side of assignments and that may contain memory operations; we let $\text{rhs} \in \text{RHS}$; (iii) $\tilde{S}$, a lattice structure that models string values that is produced by the value domain (used to resolve dynamic accesses). Intuitively, $\tilde{S}$ is an abstraction of the set of possible string values that over-approximate the string argument s in $e[s]$.

Operations marked with $(*)$ are specific to this framework; the remaining ones are standard abstract interpretation transformers, slightly modified to accommodate information exchange between the domains. The value domain must provide the following operations:

- $\text{assign}_{V^\#} : (\text{MemID} \cup \text{ID}) \times \tilde{V} \times \mathcal{V}^\# \times \{\text{true} \mid \text{false}\} \rightarrow \mathcal{V}^\#$, which assigns the abstract value denoted by the second argument to the (memory) identifier given as first argument in the abstract value state; the boolean flag selects between a strong update (overwrite) and a weak update (join with the existing value);

- $\text{assume}_{\mathcal{V}^\sharp} : \text{BE} \times \mathcal{V}^\sharp \rightarrow \mathcal{V}^\sharp$, which refines the abstract value state according to a boolean guard.
- $(*)\text{eval}_{\mathcal{V}^\sharp} : \text{SE} \times \mathcal{V}^\sharp \rightarrow \tilde{S}$, which evaluates string expressions and returns a abstract set of values used to resolve dynamic memory accesses;

Note that, while the existence of an evaluation function such as $\text{eval}_{\mathcal{V}^\sharp}$ is typical of non-relational domains, relational domains employed in whole-value analyses must also produce an abstraction of an expression's value to interact with other domains. Therefore, the framework is not restricted to non-relational value domains. For instance, a relational domain like the polyhedra domain can still produce a non relational interval reporting the range of values an expression can evaluate to. It follows that the value domain must be able to over-approximate string values to properly support the $\text{e}[s]$ instruction. If it cannot, the set $\tilde{S}$ can be instantiated with a trivial lattice (e.g., $\{\top\}$) that over-approximates all strings, and the $\text{eval}_{\mathcal{V}^\sharp}$ operator can be defined as a no-op that always returns $\top$.

Instead, the memory domain must provide the following operations:

- $\text{assign}_{\mathcal{M}^\sharp} : \text{LHS} \times \text{RHS} \times \mathcal{M}^\sharp \rightarrow \text{Sub} \times \mathcal{M}^\sharp$, which updates the abstract memory structure by assigning the second argument to the first, and returns any substitutions required to keep the value domain consistent; the first argument may be either a syntactic left-hand side (an element of LHS) or an explicit memory identifier (an element of MemID);
- $\text{assume}_{\mathcal{M}^\sharp} : \text{BE} \times \mathcal{M}^\sharp \rightarrow \mathcal{M}^\sharp$, which refines the abstract memory state according to a boolean guard;
- $(*)\text{pointsTo}_{\mathcal{M}^\sharp} : \wp(\text{ID} \cup \text{MemID}) \times \mathcal{M}^\sharp \rightarrow \wp(\text{ID} \cup \text{MemID})$, which yields the set of memory identifiers that a set of identifiers and memory identifiers may point to in the current memory state;
- $(*)\text{addrOf}_{\mathcal{M}^\sharp} : \text{ID} \times \mathcal{M}^\sharp \rightarrow \text{MemID}$, which yields the memory identifier that abstracts the address of the given program variable in the current memory state;
- $(*)\text{access}_{\mathcal{M}^\sharp} : \wp(\text{MemID}) \times \Sigma^* \times \mathcal{M}^\sharp \rightarrow \wp(\text{MemID}) \times \text{Sub} \times \mathcal{M}^\sharp$, where Σ^* is the set of string literals, which resolves field accesses starting from a set of base memory identifiers and a statically known field name to a new set of memory identifiers; since an access on the left-hand side of an assignment might define new fields, $\text{access}_{\mathcal{M}^\sharp}$ has agency to update the memory structure and return a new instance of $\mathcal{M}^\sharp$, together with any substitutions required to keep the value domain consistent;
- $(*)\text{access}_{\mathcal{M}^\sharp} : \wp(\text{MemID}) \times \tilde{S} \times \mathcal{M}^\sharp \rightarrow \wp(\text{MemID}) \times \text{Sub} \times \mathcal{M}^\sharp$ that models a field access in the same way as its previous overload, but starting from a dynamically evaluated field name that has already been abstracted to an instance of $\tilde{S}$ by the value domain;
- $(*)\text{fresh}_{\mathcal{M}^\sharp}^K : \mathcal{M}^\sharp \rightarrow \text{MemID} \times \mathcal{M}^\sharp$, which allocates fresh memory identifiers corresponding to a stack-allocated abstract location; since the memory abstraction might need update to track the new location, $\text{fresh}_{\mathcal{M}^\sharp}^K$ returns an updated instance of $\mathcal{M}^\sharp$ together with the fresh identifier, but no substitu-

tions since the location is fresh and cannot be referenced by the value domain yet;⁴

- (*) $\text{fresh}_{\mathcal{M}^\sharp}^h : \mathcal{M}^\sharp \rightarrow \text{MemID} \times \mathcal{M}^\sharp$, that performs the same role as $\text{fresh}_{\mathcal{M}^\sharp}^k$ but for heap-allocated abstract locations;
- (*) $\text{memId}_{\mathcal{M}^\sharp} : \mathcal{M}^\sharp \rightarrow \wp(\text{MemID})$, which returns the set of memory identifiers currently tracked in the given abstract memory state; used to state the coverage condition on γ_{MemID} and $\gamma_{\text{MemID}f}$ (see C1 below);
- (*) $\text{isSum}_{\mathcal{M}^\sharp} : \text{MemID} \times \mathcal{M}^\sharp \rightarrow \{\text{true} \mid \text{false}\}$, which returns **true** if the memory identifier may represent more than one concrete location (a summary node), and **false** otherwise (represent exactly one, i.e., a singleton node).

6.4 Substitutions

The memory domain may reorganize abstract memory identifiers during abstract computations, for instance by materializing (or splitting), merging and renaming locations. These identifier changes must be communicated to the value domain so the two components remain consistent.

We adopt the substitution concept from [20], and specialize it to our setting. A substitution $\text{sub} = [l_1 \rightarrow l'_1, \dots, l_n \rightarrow l'_n] \in \text{Sub}$ is a finite sequence of replacements, where each $l_j, l'_j \subseteq \text{MemID}$. A binding $l \rightarrow l'$ states that the post-state identifiers in l derive from the pre-state identifiers in l' ; identifiers in l' not appearing in any l are killed. The three canonical cases are: (i) *rename*: l and l' are both singletons; (ii) *merge*: l is a singleton, $|l'| > 1$ (e.g., $\{\iota_1\} \rightarrow \{\iota_2, \iota_3\}$ merges ι_2 and ι_3); (iii) *split/materialization*: $|l| > 1$, l' is a singleton (e.g., $\{\iota_1, \iota_2\} \rightarrow \{\iota_3\}$ splits ι_3).

When a memory domain transformer returns a substitution sub , the value domain applies it via $\text{applySub}(\text{sub}, \nu^\sharp, \mu^\sharp)$, defined as:

$$\begin{aligned} \text{applySub}(\text{sub}, \nu^\sharp, \mu^\sharp) &= \nu_n^\sharp \text{ where } \text{sub} = [l_1 \rightarrow l'_1, \dots, l_n \rightarrow l'_n], \nu_0^\sharp = \nu^\sharp \\ &\wedge \forall j \in [1, n] : \nu_j^\sharp = \sqcup_{l' \in l'_j} \{\nu_m^{\sharp'} : l_j = \{\iota_1, \dots, \iota_m\}, \nu_0^{\sharp'} = \nu_{j-1}^\sharp, \\ &\forall k \in [1, m] : \nu_k^{\sharp'} = \text{assign}_{\nu^\sharp}(\iota_k, \iota', \nu_{k-1}^{\sharp'}, \neg \text{isSum}_{\mathcal{M}^\sharp}(\iota_k, \mu^\sharp))\} \end{aligned}$$

6.5 Expression Rewriting

During abstract computations, the right-hand side of an assignment may contain memory-related sub-expressions (e.g., pointer dereferences, field accesses, object allocations) that must be rewritten to memory identifiers since the value domain cannot handle them. The function $R_{\mathcal{M}^\sharp} : \text{RHS} \times \mathcal{V}^\sharp \times \mathcal{M}^\sharp \rightarrow \wp(\tilde{V}) \times \mathcal{V}^\sharp \times \mathcal{M}^\sharp$ handles this rewriting: it traverses a right-hand-side expression, resolves every

⁴ Note that $\text{fresh}_{\mathcal{M}^\sharp}^k$ does not need to return an unused memory identifier at every call, but rather an address for a particular allocation: for instance, an allocation-site based analysis will generate one identifier for each location in the program, but will always generate the same identifier for the same location at each loop iteration.

memory-sensitive sub-expression to a set of abstract value references, and threads the updated abstract state through the process. To avoid cluttering the definition, we will pass sets in $\wp(\tilde{V})$ to functions that expect as arguments sets of $\wp(\text{MemID})$, implicitly assuming that any element of the argument that is not a memory identifier is ignored by the function. Function $R_{\mathcal{M}^\sharp}$ is defined by structural induction on the syntax of the input expression as:

$$\begin{aligned}
R_{\mathcal{M}^\sharp}(\mathbf{x}, \nu^\sharp, \mu^\sharp) &= (\{\mathbf{x}\}, \nu^\sharp, \mu^\sharp) \\
R_{\mathcal{M}^\sharp}(\mathbf{a} \mid \mathbf{b} \mid \mathbf{s}, \nu^\sharp, \mu^\sharp) &= (\{\mathbf{a} \mid \mathbf{b} \mid \mathbf{s}\}, \nu^\sharp, \mu^\sharp) \\
R_{\mathcal{M}^\sharp}(\&\mathbf{x}, \nu^\sharp, \mu^\sharp) &= (\{\text{addrOf}_{\mathcal{M}^\sharp}(\mathbf{x}, \mu^\sharp)\}, \nu^\sharp, \mu^\sharp) \\
R_{\mathcal{M}^\sharp}(*\mathbf{p}, \nu^\sharp, \mu^\sharp) &= (l_1, \nu_1^\sharp, \mu_1^\sharp) \\
&\quad \text{where } (l_1, \nu_1^\sharp, \mu_1^\sharp) = R_{\mathcal{M}^\sharp}(\mathbf{p}, \nu^\sharp, \mu^\sharp) \wedge \\
&\quad l_1 = \text{pointsTo}_{\mathcal{M}^\sharp}(l, \mu_1^\sharp) \\
R_{\mathcal{M}^\sharp}(\mathbf{p}.\mathbf{f}, \nu^\sharp, \mu^\sharp) &= (ac, \text{applySub}(\text{sub}, \nu_1^\sharp, \mu_2^\sharp), \mu_2^\sharp) \\
&\quad \text{where } (ids, \nu_1^\sharp, \mu_1^\sharp) = R_{\mathcal{M}^\sharp}(\mathbf{p}, \nu^\sharp, \mu^\sharp) \wedge \\
&\quad (ac, \text{sub}, \mu_2^\sharp) = \text{access}_{\mathcal{M}^\sharp}(ids, \mathbf{f}, \mu_1^\sharp) \\
R_{\mathcal{M}^\sharp}(\mathbf{p}[\mathbf{s}], \nu^\sharp, \mu^\sharp) &= (ac, \text{applySub}(\text{sub}, \nu_1^\sharp, \mu_2^\sharp), \mu_2^\sharp) \\
&\quad \text{where } (ids, \nu_1^\sharp, \mu_1^\sharp) = R_{\mathcal{M}^\sharp}(\mathbf{p}, \nu^\sharp, \mu^\sharp) \wedge \\
&\quad S = \text{eval}_{\mathcal{V}^\sharp}(\mathbf{s}, \nu_1^\sharp) \wedge \\
&\quad (ac, \text{sub}, \mu_2^\sharp) = \text{access}_{\mathcal{M}^\sharp}(ids, S, \mu_1^\sharp) \\
R_{\mathcal{M}^\sharp}(\{\ell\}, \nu^\sharp, \mu^\sharp) &= (\{\ell\}, \nu^\sharp, \mu_1^\sharp) \\
&\quad \text{where } (\ell, \mu_1^\sharp) = \text{fresh}_{\mathcal{M}^\sharp}^{\mathcal{K}}(\mu^\sharp) \\
R_{\mathcal{M}^\sharp}(\{\mathbf{f_i}:\mathbf{v_i}\}_{i=1}^n, \nu_0^\sharp, \mu^\sharp) &= (\{\ell\}, \nu_n^\sharp, \mu_n^\sharp) \\
&\quad \text{where } (\ell, \mu_0^\sharp) = \text{fresh}_{\mathcal{M}^\sharp}^{\mathcal{K}}(\mu^\sharp) \wedge \\
&\quad \text{for } i = 1..n : \\
&\quad (l_i, \text{sub}_i, \mu_i^\sharp) = \text{access}_{\mathcal{M}^\sharp}(\{\ell\}, \mathbf{f}_i, \mu_{i-1}^\sharp) \wedge \\
&\quad \nu_{s_i}^\sharp = \text{applySub}(\text{sub}_i, \nu_{i-1}^\sharp, \mu_i^\sharp) \wedge \\
&\quad \nu_i^\sharp = \bigsqcup_{\iota \in l_i} \text{assign}_{\mathcal{V}^\sharp}(\iota, \mathbf{v}_i, \nu_{s_i}^\sharp, \neg \text{isSum}_{\mathcal{M}^\sharp}(\iota, \mu_i^\sharp)) \\
R_{\mathcal{M}^\sharp}(\text{new } \mathbf{a} \mid \mathbf{b} \mid \mathbf{s}, \nu^\sharp, \mu^\sharp) &= (\{\ell\}, \nu_1^\sharp, \mu_1^\sharp) \\
&\quad \text{where } (\ell, \mu_1^\sharp) = \text{fresh}_{\mathcal{M}^\sharp}^{\mathcal{H}}(\mu^\sharp) \wedge \\
&\quad \nu_1^\sharp = \text{assign}_{\mathcal{V}^\sharp}(\ell, \mathbf{a} \mid \mathbf{b} \mid \mathbf{s}, \nu^\sharp, \neg \text{isSum}_{\mathcal{M}^\sharp}(\ell, \mu^\sharp)) \\
R_{\mathcal{M}^\sharp}(\text{new } \{\ell\}, \nu^\sharp, \mu^\sharp) &= (\{\ell\}, \nu^\sharp, \mu_1^\sharp) \\
&\quad \text{where } (\ell, \mu_1^\sharp) = \text{fresh}_{\mathcal{M}^\sharp}^{\mathcal{H}}(\mu^\sharp)
\end{aligned}$$

$$\begin{aligned}
R_{\mathcal{M}^\sharp}(\text{new } \{\mathbf{f_i}:\mathbf{v_i}\}_{i=1}^n, \nu_0^\sharp, \mu^\sharp) &= (\{\ell\}, \nu_n^\sharp, \mu_n^\sharp) \\
&\text{where } (\ell, \mu_0^\sharp) = \text{fresh}_{\mathcal{M}^\sharp}^H(\mu^\sharp) \wedge \\
&\text{for } i = 1..n : \\
&\quad (\mathbf{l}_i, \text{sub}_i, \mu_i^\sharp) = \text{access}_{\mathcal{M}^\sharp}(\{\ell\}, \mathbf{f}_i, \mu_{i-1}^\sharp) \wedge \\
&\quad \nu_{s_i}^\sharp = \text{applySub}(\text{sub}_i, \nu_{i-1}^\sharp, \mu_i^\sharp) \wedge \\
&\quad \nu_i^\sharp = \bigsqcup_{\iota \in \mathbf{l}_i} \text{assign}_{\mathcal{V}^\sharp}(\iota, \mathbf{v}_i, \nu_{s_i}^\sharp, \neg \text{isSum}_{\mathcal{M}^\sharp}(\iota, \mu_i^\sharp))
\end{aligned}$$

6.6 Statement Semantics

The abstract semantics is defined to abstract the split semantics while operating over abstract states. For each statement $\mathbf{st}$, we define an abstract transfer function $\llbracket \mathbf{st} \rrbracket^\sharp : \sigma^\sharp \rightarrow \sigma^\sharp$. The semantics of assignment statements is defined as follows:

$$\begin{aligned}
&(\text{sub}_m, \mu_1^\sharp) = \text{assign}_{\mathcal{M}^\sharp}(\text{lhs}, \text{rhs}, \mu^\sharp) \quad \nu_m^\sharp = \text{applySub}(\text{sub}_m, \nu^\sharp, \mu_1^\sharp) \\
&(\text{id}_{s_r}, \nu_r^\sharp, \mu_r^\sharp) = R_{\mathcal{M}^\sharp}(\text{rhs}, \nu_m^\sharp, \mu_1^\sharp) \quad (\text{id}_{s_l}, \nu_l^\sharp, \mu_l^\sharp) = R_{\mathcal{M}^\sharp}(\text{lhs}, \nu_r^\sharp, \mu_r^\sharp) \\
&(\text{assign}) \quad \frac{\nu_2^\sharp = \bigsqcup_{\text{id}_l \in \text{id}_{s_l}, \text{id}_r \in \text{id}_{s_r}} \text{assign}_{\mathcal{V}^\sharp}(\text{id}_l, \text{id}_r, \nu_l^\sharp, \neg \text{isSum}_{\mathcal{M}^\sharp}(\text{id}_l, \mu_l^\sharp))}{\llbracket \text{lhs} = \text{rhs} \rrbracket^\sharp(\nu^\sharp, \mu^\sharp) = (\nu_2^\sharp, \mu_l^\sharp)}
\end{aligned}$$

Note that we can call $R_{\mathcal{M}^\sharp}$ on lhs since $\text{LHS} \subseteq \text{RHS}$. Also, we can call $\text{assign}_{\mathcal{V}^\sharp}$ on id_l since the function $R_{\mathcal{M}^\sharp}$ always returns a set of memory identifiers MemID when called on a lhs expression. Finally, the assignment on id_l is strong if it is not a summary node, and weak otherwise.

The abstract counterpart of $\text{ASM}(\mathbf{b})$ refines both domain components independently using the $\text{assume}_{\mathcal{V}^\sharp}$ and $\text{assume}_{\mathcal{M}^\sharp}$ operations provided by each domain:

$$(\text{assume}) \quad \frac{\nu_1^\sharp = \text{assume}_{\mathcal{V}^\sharp}(\mathbf{b}, \nu^\sharp) \quad \mu_1^\sharp = \text{assume}_{\mathcal{M}^\sharp}(\mathbf{b}, \mu^\sharp)}{\llbracket \text{ASM}(\mathbf{b}) \rrbracket^\sharp(\nu^\sharp, \mu^\sharp) = (\nu_1^\sharp, \mu_1^\sharp)}$$

Note that, since $\mathbf{b}$ cannot contain memory expressions, function $R_{\mathcal{M}^\sharp}$ does not need to be applied in assume operations.

Example 3 (Value–Memory Interaction). Similarly to Example 2, consider the program $\mathbf{p} = \text{new } 5; \text{tmp} = *p; \mathbf{y} = \text{tmp} + 1$. We assume the memory domain is the Andersen-style points-to analysis (Section 7.2) and the value domain is constant propagation; Table 2 shows the resulting abstract state after each statement. To show the interaction between the value and memory domains, consider the second statement $\text{tmp} = *p$, where the abstract assignment rule (Section 6) proceeds as follows.

1. $\text{assign}_{\mathcal{M}^\sharp}(\text{tmp}, *p, \mu^\sharp)$: a load does not change pointer structure, so it returns an empty substitution and $\mu^\sharp$ unchanged.

Table 2. Abstract state at each program point. The memory domain determines *which* location is read (ι_ℓ , step 2); the value domain determines *what* is stored there (5, step 4).

After statement	$\nu^\#$	$\mu^\#$
p = new 5	$[\iota_\ell \mapsto 5]$	$[p \mapsto \{\iota_\ell\}]$
tmp = *p	$[\iota_\ell \mapsto 5, \text{tmp} \mapsto 5]$	$[p \mapsto \{\iota_\ell\}]$
y = tmp + 1	$[\iota_\ell \mapsto 5, \text{tmp} \mapsto 5, y \mapsto 6]$	$[p \mapsto \{\iota_\ell\}]$

2. $R_{\mathcal{M}^\#}(*p, \nu^\#, \mu^\#)$: the dereference rule evaluates $R_{\mathcal{M}^\#}(p, \nu^\#, \mu^\#) = (\{p\}, \nu^\#, \mu^\#)$ first, then queries $\text{pointsTo}_{\mathcal{M}^\#}(\{p\}, \mu^\#) = \{\iota_\ell\}$. The memory domain supplies the identifier to read: $ids_r = \{\iota_\ell\}$.
3. $R_{\mathcal{M}^\#}(\text{tmp}, \nu^\#, \mu^\#) = (\{\text{tmp}\}, \nu^\#, \mu^\#)$: the variable rule gives $ids_l = \{\text{tmp}\}$.
4. $\text{assign}_{\mathcal{V}^\#}(\text{tmp}, \iota_\ell, \nu^\#, \neg \text{isSum}_{\mathcal{M}^\#}(\text{tmp}, \mu^\#))$: since **tmp** is a concrete variable (not a summary node), $\text{isSum}_{\mathcal{M}^\#}(\text{tmp}, \mu^\#) = \text{false}$ and the assignment is strong; constant propagation reads $\nu^\#(\iota_\ell) = 5$ and sets $\nu^\#(\text{tmp}) \leftarrow 5$.

For **y = tmp + 1** all four steps follow the same structure, with the memory domain playing no role:

1. $\text{assign}_{\mathcal{M}^\#}(y, \text{tmp}+1, \mu^\#)$: an arithmetic assignment does not change pointer structure; returns an empty substitution and $\mu^\#$ unchanged.
2. $R_{\mathcal{M}^\#}(\text{tmp}+1, \nu^\#, \mu^\#)$: the scalar rule returns the expression as-is, so $ids_r = \{\text{tmp}+1\}$.
3. $R_{\mathcal{M}^\#}(y, \nu^\#, \mu^\#) = (\{y\}, \nu^\#, \mu^\#)$: the variable rule gives $ids_l = \{y\}$.
4. $\text{assign}_{\mathcal{V}^\#}(y, \text{tmp}+1, \nu^\#, \neg \text{isSum}_{\mathcal{M}^\#}(y, \mu^\#))$: the operation $\text{isSum}_{\mathcal{M}^\#}(y, \mu^\#)$ returns **false** because **y** is a concrete variable, resulting in a strong assignment; constant propagation evaluates $\nu^\#(\text{tmp}) + 1 = 6$ and sets $\nu^\#(y) \leftarrow 6$.

6.7 Soundness

The soundness of our abstract domain can be established by assuming some conditions over the transformers provided by the value and memory domains. Specifically, we assume two conditions on how memory identifiers are concretized in order to prove the soundness of the analysis. These are adapted from C1 and C3 of [20].

- C1. γ_{MemID} and γ_{MemIDf} must together cover all memory identifiers tracked in the concretized memory state. Formally, $\forall \mu^\# \in \mathcal{M}^\#, \forall (\mu, \gamma_{\text{MemID}}, \gamma_{\text{MemIDf}}) \in \gamma_{\mathcal{M}^\#}(\mu^\#) : \text{dom}(\gamma_{\text{MemID}}) \cup \text{dom}(\gamma_{\text{MemIDf}}) = \text{memId}_{\mathcal{M}^\#}(\mu^\#)$.
- C2. Different memory identifiers represent disjoint portions of memory, that is, $\text{dom}(\gamma_{\text{MemID}}) \cap \text{dom}(\gamma_{\text{MemIDf}}) = \emptyset$ (since intuitively one concretizes locations and the other concretizes object fields) and for any $\iota_1, \iota_2 \in \text{dom}(\gamma_{\text{MemIDf}})$, $\gamma_{\text{MemIDf}}(\iota_1) \cap \gamma_{\text{MemIDf}}(\iota_2) = \emptyset$ (since different identifiers must represent disjoint fields).

For the value abstraction, we employ the usual requirements of Abstract Interpretation: the $\text{assign}_{\mathcal{V}^\sharp}$ must correctly over-approximate the effect of assignments, the $\text{assume}_{\mathcal{V}^\sharp}$ must correctly refine the abstract state according to boolean guards, and $\text{eval}_{\mathcal{V}^\sharp}$ must over-approximate the possible values of string expressions. The same requirements are also employed for $\text{assign}_{\mathcal{M}^\sharp}$ and $\text{assume}_{\mathcal{M}^\sharp}$ in the memory domain.

To state requirements on framework-specific operators uniformly, we define for any $\iota \in \text{dom}(\gamma_{\text{MemID}}) \cup \text{dom}(\gamma_{\text{MemID}_f})$, with $(\mu, \gamma_{\text{MemID}}, \gamma_{\text{MemID}_f}) \in \gamma_{\mathcal{M}^\sharp}(\mu^\sharp)$:

$$\text{locs}(\iota) \triangleq \begin{cases} \gamma_{\text{MemID}}(\iota) & \text{if } \iota \in \text{dom}(\gamma_{\text{MemID}}), \\ \{a : (a, f) \in \gamma_{\text{MemID}_f}(\iota)\} & \text{if } \iota \in \text{dom}(\gamma_{\text{MemID}_f}). \end{cases}$$

The framework-specific operators must satisfy the following:

- M1. $\text{pointsTo}_{\mathcal{M}^\sharp}$ must correctly include all possible points-to targets of the given set of identifiers: $\forall I \subseteq \text{ID} \cup \text{MemID}, \mu^\sharp \in \mathcal{M}^\sharp : \bigcup_{i' \in \text{pointsTo}_{\mathcal{M}^\sharp}(I, \mu^\sharp)} \text{locs}(i') \supseteq \{a : \exists i \in I : a \in \text{locs}(i)\}$;
- M2. $\text{addrOf}_{\mathcal{M}^\sharp}$ must correctly produce the memory identifier that abstracts the address of the given program variable: $\forall x \in \text{ID}, \mu^\sharp \in \mathcal{M}^\sharp : \text{addrOf}_{\mathcal{M}^\sharp}(x, \mu^\sharp) = i$ where i is the unique memory identifier such that $\text{locs}(i) = \{a_x\}$;
- M3. both overloads of $\text{access}_{\mathcal{M}^\sharp}$ must correctly resolve field accesses starting from a set of base memory identifiers and a statically known field name: $\forall I \subseteq \text{MemID}, f \in \Sigma^*, \mu^\sharp \in \mathcal{M}^\sharp : \text{access}_{\mathcal{M}^\sharp}(I, f, \mu^\sharp) = (I', \text{sub}, \mu^{\sharp'})$ where I' is a set of memory identifiers such that $\bigcup_{i \in I'} \text{locs}(i) \supseteq \{a' : (\mu, \gamma_{\text{MemID}}, \gamma_{\text{MemID}_f}) \in \gamma_{\mathcal{M}^\sharp}(\mu^{\sharp'}) \wedge \exists i \in I, a \in \text{locs}(i) : (a, f, a') \in \mu\}$; a similar condition must hold for the second overload, where f is taken from the concretization of $\tilde{S}$; it is also required that both $\text{access}_{\mathcal{M}^\sharp}$ overloads do not change what was already tracked by the domain (i.e., $\mu^\sharp \sqsubseteq_{\mathcal{M}^\sharp} \mu^{\sharp'}$).
- M4. both $\text{fresh}_{\mathcal{M}^\sharp}^{\mathcal{K}}$ and $\text{fresh}_{\mathcal{M}^\sharp}^{\mathcal{H}}$ must produce fresh memory identifiers that do not overlap with any existing identifier in the input memory state, and must return an updated memory state that tracks the new identifier; formally, $\forall \mu^\sharp \in \mathcal{M}^\sharp : \text{fresh}_{\mathcal{M}^\sharp}^{\mathcal{K}}(\mu^\sharp) = (\iota, \mu^{\sharp'})$ and $\text{fresh}_{\mathcal{M}^\sharp}^{\mathcal{H}}(\mu^\sharp) = (\iota, \mu^{\sharp''})$ where $\iota \notin \text{memId}_{\mathcal{M}^\sharp}(\mu^\sharp)$, $\text{memId}_{\mathcal{M}^\sharp}(\mu^{\sharp'}) = \text{memId}_{\mathcal{M}^\sharp}(\mu^{\sharp''}) = \text{memId}_{\mathcal{M}^\sharp}(\mu^\sharp) \cup \{\iota\}$, and $\mu^{\sharp'}$ and $\mu^{\sharp''}$ are consistent with $\mu^\sharp$ on all identifiers in $\text{memId}_{\mathcal{M}^\sharp}(\mu^\sharp)$ (i.e., $\mu^\sharp \sqsubseteq_{\mathcal{M}^\sharp} \mu^{\sharp'}$ and $\mu^\sharp \sqsubseteq_{\mathcal{M}^\sharp} \mu^{\sharp''}$).
- M5. predicate $\text{isSum}_{\mathcal{M}^\sharp}$ must correctly identify summary nodes and singletons; formally, $\forall \iota \in \text{dom}(\gamma_{\text{MemID}}) : \text{isSum}_{\mathcal{M}^\sharp}(\iota, \mu^\sharp) = \text{false} \implies \|\gamma_{\text{MemID}}(\iota)\| = 1$, i.e., if ι is not a summary node then it concretizes to exactly one heap address.
- M6. following [20], we expect all substitutions produced by the memory domain to be coherent with the modifications that happened therein [20, Prop. 4.1].

Our framework uses two auxiliary functions in its semantics: applySub and $\text{R}_{\mathcal{M}^\sharp}$. The former is sound by [20, Lem. C.1], since M5 holds and $\text{assign}_{\mathcal{V}^\sharp}$ is sound by assumption, while the latter is novel. As formalized in the version of this paper

containing the technical report [9], $R_{\mathcal{M}^\sharp}$ is sound, that is, the values computed by the expression passed as its argument are over-approximated by the set of values computed by the rewritten expressions.

In general, soundness can be established by showing that $\forall \sigma^\sharp \in \mathcal{S}^\sharp, \text{st} \in \text{STMT} : \llbracket \text{st} \rrbracket(\gamma_{\mathcal{S}^\sharp}(\sigma^\sharp)) \subseteq \gamma_{\mathcal{S}^\sharp}(\llbracket \text{st} \rrbracket^\sharp(\sigma^\sharp))$. Intuitively, every concrete behavior admitted by a split state is also admitted by its abstract counterpart after executing the same statement.

Theorem 3 (Soundness of the Abstract Semantics). *If conditions C1–C2 and M1–M6 hold, then the abstract semantics is sound with respect to the split semantics. Concretely, for any abstract state $\sigma^\sharp = (\nu^\sharp, \mu^\sharp) \in \mathcal{S}^\sharp$, the following must hold:*

- $\llbracket \text{lhs} = \text{rhs} \rrbracket(\gamma_{\mathcal{S}^\sharp}(\sigma^\sharp)) \subseteq \gamma_{\mathcal{S}^\sharp}(\llbracket \text{lhs} = \text{rhs} \rrbracket^\sharp(\sigma^\sharp))$, and
- $\llbracket \text{ASM}(\text{b}) \rrbracket(\gamma_{\mathcal{S}^\sharp}(\sigma^\sharp)) \subseteq \gamma_{\mathcal{S}^\sharp}(\llbracket \text{ASM}(\text{b}) \rrbracket^\sharp(\sigma^\sharp))$.

Proof in the technical report [9].

6.8 Practical Benefits of Modularity

The modular design of the framework delivers three concrete practical benefits.

- *Reusability*: an existing value domain or memory domain can be plugged into the framework at the cost of adapting it to the common interface. As the instantiation of Section 7 shows, this corresponds to implementing a modest set of operators ($\text{assign}_{\mathcal{V}^\sharp}$, $\text{assume}_{\mathcal{V}^\sharp}$, and $\text{eval}_{\mathcal{V}^\sharp}$ for the value side; $\text{assign}_{\mathcal{M}^\sharp}$, $\text{assume}_{\mathcal{M}^\sharp}$, $\text{memld}_{\mathcal{M}^\sharp}$, $\text{pointsTo}_{\mathcal{M}^\sharp}$, $\text{addrOf}_{\mathcal{M}^\sharp}$, $\text{access}_{\mathcal{M}^\sharp}$, $\text{fresh}_{\mathcal{M}^\sharp}^c$, and $\text{fresh}_{\mathcal{M}^\sharp}^h$ for the memory side), each of which usually maps naturally to operations the domain already performs.
- *Tunable precision and efficiency*: both the value and memory domains can be swapped for a cheaper or a more precise abstraction independently of the other component; the example at the end of this Section demonstrates this for constants vs. intervals vs. constant sets on the same program.
- *Compositional soundness*: each domain only needs to satisfy a set of local conditions (Section 6.7, Theorem 3); if both domains satisfy them, the combined analysis is automatically sound, i.e., there is no need to prove the correctness of the combination from scratch since the framework guarantees it by construction.

Implementation-wise, a modular implementation following our framework encourages reuse and testing, and removes the need for defining custom combinations for each pair of domains.

Compared to running the memory and value analyses in a Cartesian product (or in isolation), the framework presents a clear tradeoff. On the one hand, the value domain tracks information not only on the variables but also on the memory locations. Since the complexity of value domains usually depends on the number of variables, our approach increases this value by adding all possible

memory identifiers, thus producing a less efficient analysis. However, any analysis that combines memory and value information will incur in this additional cost. On the other hand, it allows tracking of value information on memory locations, something that would not be possible in the Cartesian product (or running the analyses in isolation), thus sensibly improving the precision of the analysis.

Example 4 (Modularity in Action). The following example showcases the modularity of our approach by plugging a different value domain into the framework, while keeping the memory domain fixed, directly affects the precision of the result, without any other change to the analysis.

Program. Consider the μLL program below. It allocates a single heap cell initialised to 0, non-deterministically overwrites it with either 1 or 3, and then loads the result into `q`.

Listing 3. Program for the three-domain modularity example.

```

1  p = new 0;
2  if nd {
3      *p = 1;
4  } else {
5      *p = 3;
6  }
7  q = *p;
```

Memory Domain. To analyze this program, we use the Andersen-style points-to domain of Section 7.2. The single `new` expression at line 1 gives rise to one abstract memory identifier $\iota_\ell \in \text{MemID}$. After line 1, the memory state is $\mu^\sharp = [\text{p} \mapsto \{\iota_\ell\}]$ and this state is preserved unchanged throughout the if-else block and the final load: neither branch adds or removes pointer targets. The value stored at ι_ℓ is tracked entirely by the value domain.

Value Domain 1 – Constant Propagation (CP). The CP domain represents each identifier by either a known integer constant c or the top element $\top$ (value unknown). The join of two distinct constants is $\top$.

- After line 1: $\nu^\sharp(\iota_\ell) = 0$.
- After line 3 (then-branch): $\nu^\sharp(\iota_\ell) = 1$.
- After line 5 (else-branch): $\nu^\sharp(\iota_\ell) = 3$.
- After the join: $\nu^\sharp(\iota_\ell) = 1 \sqcup_{\text{CP}} 3 = \top$.
- After line 7: $\nu^\sharp(\text{q}) = \top$.

Constant propagation cannot bound the value of `q` in any way.

Value Domain 2 – Intervals (Int). The interval domain represents each identifier by a range $[l, u]$. The join of two singletons $[1, 1]$ and $[3, 3]$ is $[1, 3]$.

- After line 1: $\nu^\sharp(\iota_\ell) = [0, 0]$.

- After line 3 (then-branch): $\nu^\sharp(\iota_\ell) = [1, 1]$.
- After line 5 (else-branch): $\nu^\sharp(\iota_\ell) = [3, 3]$.
- After the join: $\nu^\sharp(\iota_\ell) = [1, 1] \sqcup_{\text{Int}} [3, 3] = [1, 3]$.
- After line 7: $\nu^\sharp(\mathbf{q}) = [1, 3]$.

The interval analysis establishes $\mathbf{q} \in [1, 3]$, in particular $\mathbf{q} \geq 1$ and $\mathbf{q} \leq 3$.

Value Domain 3 – Constant Sets (CSet_k). We use the bounded variant of the constant-set domain [32], which represents each identifier by a finite set of integer constants of cardinality at most k , or $\top$ otherwise. Join and widening both collapse to $\top$ when $|A \cup B| > k$; within the bound, join is set union. In this example, having $k \geq 2$ is sufficient to capture the exact set of possible values.

- After line 1: $\nu^\sharp(\iota_\ell) = \{0\}$.
- After line 3 (then-branch): $\nu^\sharp(\iota_\ell) = \{1\}$.
- After line 5 (else-branch): $\nu^\sharp(\iota_\ell) = \{3\}$.
- After the join: $\nu^\sharp(\iota_\ell) = \{1\} \sqcup_{\text{CSet}} \{3\} = \{1, 3\}$.
- After line 7: $\nu^\sharp(\mathbf{q}) = \{1, 3\}$.

The constant-set analysis yields the exact set of possible values, establishing $\mathbf{q} \in \{1, 3\}$ precisely.

Table 3. Abstract states at key points for the three instantiations. The memory state $\mu^\sharp$ is identical throughout; only the value domain changes the precision of the result.

Program point	$\mu^\sharp$	$\nu_{\text{CP}}^\sharp(\iota_\ell)$	$\nu_{\text{Int}}^\sharp(\iota_\ell)$	$\nu_{\text{CSet}}^\sharp(\iota_\ell)$
After line 1	$[\mathbf{p} \mapsto \{\iota_\ell\}]$	0	$[0, 0]$	$\{0\}$
After line 3 (then)	$[\mathbf{p} \mapsto \{\iota_\ell\}]$	1	$[1, 1]$	$\{1\}$
After line 5 (else)	$[\mathbf{p} \mapsto \{\iota_\ell\}]$	3	$[3, 3]$	$\{3\}$
After the join	$[\mathbf{p} \mapsto \{\iota_\ell\}]$	$\top$	$[1, 3]$	$\{1, 3\}$
After line 7 ($\nu^\sharp(\mathbf{q})$)	$[\mathbf{p} \mapsto \{\iota_\ell\}]$	$\top$	$[1, 3]$	$\{1, 3\}$

The memory state $\mu^\sharp$ is identical in all three analyses. Swapping the value domain requires no change to the memory domain or to the abstract semantics of the framework; it affects only the precision of the inferred value information, as summarised in Table 3. This demonstrates concretely that the modular design of the framework allows independent selection of value and memory abstractions.

7 Instantiations

This section instantiates the framework of Section 6 with concrete abstract domains. Note that, as this framework is an extension of [20], all instantiations supported by the latter can also be employed in our framework. To showcase the differences in the integration of abstract domains w.r.t. [20], we present two

instantiations: classical *non-relational numerical domains* [32] as value domains $\mathcal{V}^\sharp$, and Andersen-style points-to analysis [1] as the memory domain $\mathcal{M}^\sharp$. Together they yield a concrete, sound analysis for μLL .

7.1 Non-relational Numerical Domains

For each $x \in \text{ID}$ and $\iota \in \text{MemID}$, the domain maintains a non-relational abstract value $v^\sharp$ over-approximating the set of numerical values that an identifier may hold at run-time. Non-relational means that the domain tracks information on each identifier independently, that is, without tracking (e.g., symbolic) relations between different identifiers.

Abstract Values. We assume [32, Sect. 4.1.1] that the domain defines a set of abstract values $V^\sharp$ together with a partial order $\sqsubseteq_V$, least upper bound $\sqcup_V$, greatest lower bound $\sqcap_V$, and widening ∇_V operators, and concretization γ_V and abstraction α_V functions. These operators form a complete lattice, and a Galois connection with a concrete domain representing the set of numerical values whose lattice structure is defined with set operators. In addition, we assume that sound abstract operators for standard arithmetic operations (such as evaluating constant values, addition, multiplication, etc.) are provided.

Abstract States. The abstract state maps each identifier to an abstract value. The various operators are defined as the point-wise lifting of the operators over abstract values [32, Sect. 4.1.2].

Abstract Semantics. First of all, the non-relational abstract domain defines an $\widehat{\text{eval}}(\text{expr}, v^\sharp)$ function that evaluates expressions. This relies on the sound abstract operators defined by abstract values. Assignments then link the value resulting from the evaluation of the expression to the assigned identifier; when the boolean parameter of the `assignv` is `true`, the assignment is a strong update (overwrite), and when `false`, a weak update (join, $\sqcup$, with the previous abstract value) [32, Sect. 4.1.3].

Examples. In the literature, several non-relational numerical abstract domains have been formalized. The most popular ones are:

- the Sign domain [32, Sect. 4.2], tracking the sign of each identifier (e.g., if it is positive, negative, or zero);
- the constant propagation domain [32, Sect. 4.3], tracking if an identifier always has a constant numerical value;
- the interval domain [32, Sect. 4.5] tracking the minimum and maximum value of each identifier; and
- the congruence domain [24] tracking if the value assigned to an identifier is always congruent to n modulo m .

All these domains define the abstract values described in this Section and can therefore be used in our framework.

7.2 Andersen-Style Points-To Analysis

We instantiate $\mathcal{M}^\sharp$ with an Andersen-style points-to analysis [1]. For each abstract memory identifier in **MemID**, the domain tracks the set of identifiers it may point to. Since the framework is flow-sensitive, the points-to sets are threaded through the abstract semantics, growing monotonically at each statement and being joined at merge points; this subsumes the precision of the original flow-insensitive Andersen analysis.

Identifier Naming Convention. We fix a static naming scheme for **MemID** that makes the analysis deterministic and finitely bounded:

- For each program variable $x \in \text{ID}$, a distinguished *base identifier* $\iota_x \in \text{MemID}$ represents the abstract memory location of x .
- For each syntactic allocation site ℓ (a **new**-expression or object literal occurring in the program text), a distinguished base identifier $\iota_\ell \in \text{MemID}$ represents all objects born at that site (*allocation-site abstraction*): every execution of the same **new** collapses to the same abstract identifier.
- For each base identifier ι and field name $f \in \text{ID}$, a distinguished *field identifier* $\iota_{\iota,f} \in \text{MemID}$ represents the abstract memory cell for field f of the object ι .

All such identifiers are pairwise distinct. We write $\text{MemID}^b \subset \text{MemID}$ for the set of base identifiers and $\text{MemID}^f \subset \text{MemID}$ for the set of field identifiers; these two sets are disjoint. The total set of identifiers is finite (bounded by $|\text{ID}|$ plus the number of allocation sites times the number of field names), so no widening is needed for the memory domain.

Domain Definition. The abstract memory domain is the set of finite partial functions

$$\mathcal{M}^{\sharp\text{PT}} = (\text{MemID} \cup \text{ID}) \rightarrow \wp(\text{MemID}).$$

An element $\mu^\sharp \in \mathcal{M}^{\sharp\text{PT}}$ maps each currently *tracked* identifier (either a program variable or a memory identifier) to its *points-to set*: the over-approximation of the concrete addresses that the corresponding identifier may refer to. A non-tracked identifier $\iota \notin \text{dom}(\mu^\sharp)$ is treated as having an empty points-to set (it does not hold a pointer, or it has not yet been allocated).

The partial order and lattice operators are pointwise:

$$\begin{aligned} \mu_1^\sharp \sqsubseteq \mu_2^\sharp &\iff \text{dom}(\mu_1^\sharp) \subseteq \text{dom}(\mu_2^\sharp) \wedge \forall \iota \in \text{dom}(\mu_1^\sharp) : \mu_1^\sharp(\iota) \subseteq \mu_2^\sharp(\iota), \\ (\mu_1^\sharp \sqcup \mu_2^\sharp)(\iota) &= \mu_1^\sharp(\iota) \cup \mu_2^\sharp(\iota) \quad (\text{treating missing entries as } \emptyset), \\ \perp_{\mathcal{M}^{\sharp\text{PT}}} &= \emptyset \quad (\text{empty domain}), \\ \top_{\mathcal{M}^{\sharp\text{PT}}}(\iota) &= \text{MemID} \quad \text{for all } \iota \in \text{MemID}. \end{aligned}$$

Since **MemID** is finite, every ascending chain stabilises; widening reduces to join: $\mu_1^\sharp \nabla \mu_2^\sharp = \mu_1^\sharp \sqcup \mu_2^\sharp$.

Abstract Memory State and Concretization. The concretization $\gamma_{\mathcal{M}^\sharp} : \mathcal{M}^{\sharp\text{PT}} \rightarrow \wp(\mathcal{M} \times \gamma_{\text{MemID}} \times \gamma_{\text{MemID}_f})$ maps each abstract state to the set of concrete memory structures consistent with it.

For a given $\mu^\sharp \in \mathcal{M}^{\sharp\text{PT}}$, a triple $(\mu, \gamma_{\text{MemID}}, \gamma_{\text{MemID}_f}) \in \gamma_{\mathcal{M}^\sharp}(\mu^\sharp)$ must satisfy:

1. *Separation:* γ_{MemID} is defined on $\text{base}(\mu^\sharp) \triangleq \text{dom}(\mu^\sharp) \cap \text{MemID}^b$ and maps each base identifier to a non-empty set of concrete addresses: $\gamma_{\text{MemID}}(\iota) \subseteq \text{ADDR}^+$. γ_{MemID_f} is defined on $\text{field}(\mu^\sharp) \triangleq \text{dom}(\mu^\sharp) \cap \text{MemID}^f$ and maps each field identifier to a set of concrete field locations: $\gamma_{\text{MemID}_f}(\iota) \subseteq (\text{ADDR}_H \cup \text{ADDR}_R) \times \text{ID}$.
2. *Soundness:* for every tracked identifier $\iota \in \text{dom}(\mu^\sharp)$ and every concrete location $\ell \in \text{locs}(\iota)$ (where locs is as in condition C2 of Section 6), if the concrete memory holds a pointer at ℓ (i.e., ℓ appears in $\text{dom}(\mu_{ih})$ or $\text{dom}(\mu_f)$), then the pointed-to concrete address belongs to $\bigcup_{\iota' \in \mu^\sharp(\iota)} \text{locs}(\iota')$:

$$\mu_{ih}(\ell) \in \bigcup_{\iota' \in \mu^\sharp(\iota)} \text{locs}(\iota') \quad \text{or} \quad \mu_f(\ell, f) \in \bigcup_{\iota' \in \mu^\sharp(\iota)} \text{locs}(\iota')$$

(whichever applies to ℓ).

We now verify the two structural conditions required by the framework.

Lemma 2 (C1 for $\mathcal{M}^{\sharp\text{PT}}$). *For every $\mu^\sharp \in \mathcal{M}^{\sharp\text{PT}}$ and $(\mu, \gamma_{\text{MemID}}, \gamma_{\text{MemID}_f}) \in \gamma_{\mathcal{M}^\sharp}(\mu^\sharp)$: $\text{dom}(\gamma_{\text{MemID}}) \cup \text{dom}(\gamma_{\text{MemID}_f}) = \text{memld}_{\mathcal{M}^\sharp}(\mu^\sharp)$. Proof in the version of this paper containing the technical report [9].*

Lemma 3 (C2 for $\mathcal{M}^{\sharp\text{PT}}$). *For every $\mu^\sharp \in \mathcal{M}^{\sharp\text{PT}}$ and $(\mu, \gamma_{\text{MemID}}, \gamma_{\text{MemID}_f}) \in \gamma_{\mathcal{M}^\sharp}(\mu^\sharp)$, and distinct $\iota_1, \iota_2 \in \text{dom}(\gamma_{\text{MemID}}) \cup \text{dom}(\gamma_{\text{MemID}_f})$: $\text{locs}(\iota_1) \cap \text{locs}(\iota_2) = \emptyset$; moreover, for field identifiers with the same parent base, $\gamma_{\text{MemID}_f}(\iota_1) \cap \gamma_{\text{MemID}_f}(\iota_2) = \emptyset$ as sets of (a, f) pairs. Proof in the technical report [9].*

Lemma 4 (Monotonicity of $\gamma_{\mathcal{M}^\sharp}$). *If $\mu_1^\sharp \sqsubseteq \mu_2^\sharp$ then $\gamma_{\mathcal{M}^\sharp}(\mu_1^\sharp) \subseteq \gamma_{\mathcal{M}^\sharp}(\mu_2^\sharp)$. Proof in the technical report [9].*

Required Operations.

$\text{memld}_{\mathcal{M}^\sharp}$. The set of memory identifiers known to the domain can be formalized as $\text{memld}_{\mathcal{M}^\sharp}(\mu^\sharp) = (\text{dom}(\mu^\sharp) \cup \bigcup \text{codom}(\mu^\sharp)) \cap \text{MemID}$.

$\text{fresh}_{\mathcal{M}^\sharp}^{\mathcal{K}}$ and $\text{fresh}_{\mathcal{M}^\sharp}^{\mathcal{H}}$. For an allocation site ℓ (determined by the program point of the call), both functions return the canonical identifier ι_ℓ :

$$\text{fresh}_{\mathcal{M}^\sharp}^{\mathcal{K}}(\mu^\sharp) = \text{fresh}_{\mathcal{M}^\sharp}^{\mathcal{H}}(\mu^\sharp) = \iota_\ell,$$

extending the domain of $\mu^\sharp$ with $\iota_\ell \mapsto \emptyset$ if not already present.

$\text{isSum}_{\mathcal{M}^\sharp}$. Every node in the Andersen points-to graph is a summary node, as the analysis does not distinguish between different concrete objects allocated at the same site. Thus, $\text{isSum}_{\mathcal{M}^\sharp}(\iota) = \text{true}$ for all $\iota \in \text{MemID}$. Note that the analysis could easily be improved by (i) generating strong identifiers when a location is first created (i.e., it is not already part of the points-to graph), and (ii) turning the strong identifier into a summary one (by updating the internal mapping and producing the appropriate substitution) when a second object is allocated at the same site.

$\text{addrOf}_{\mathcal{M}^\sharp}$. Computing the set of identifiers that may point to a given identifier ι is straightforward:

$$\text{addrOf}_{\mathcal{M}^\sharp}(x, \mu^\sharp) = \begin{cases} \{\iota_x\} \cup \{\iota \in \text{dom}(\mu^\sharp) \mid x \in \mu^\sharp(\iota)\} & \text{if } x \in \text{ID}; \\ \{\iota \in \text{dom}(\mu^\sharp) \mid x \in \mu^\sharp(\iota)\} & \text{otherwise.} \end{cases}$$

That is, if x is a program variable, we include its base identifier ι_x . Then, we also include any identifier ι in the domain of $\mu^\sharp$ whose points-to set contains x .

$\text{pointsTo}_{\mathcal{M}^\sharp}$. As the Andersen points-to graph is represented by $\mu^\sharp$ itself, this function simply returns the points-to set of the given identifier:

$$\text{pointsTo}_{\mathcal{M}^\sharp}(\iota, \mu^\sharp) = \mu^\sharp(\iota).$$

$\text{access}_{\mathcal{M}^\sharp}$. For a set of base identifiers $\mathbf{l}$ and a statically known field name f :

$$\text{access}_{\mathcal{M}^\sharp}(\mathbf{l}, f, \mu^\sharp) = \left(\bigcup_{\iota \in \mathbf{l}} \{\iota_{\iota, f}\}, \square, \mu_{\text{ext}}^\sharp \right),$$

where $\mu_{\text{ext}}^\sharp$ extends $\mu^\sharp$ with $\iota_{\iota, f} \mapsto \emptyset$ for each ι whose field identifier is not yet tracked. For a lattice element $S \in \tilde{S}$:

- If S concretizes to a finite set of field names, apply the static case to each;
- If $S = \top_{\tilde{S}}$ (unknown selector), return all field identifiers of all objects in $\mathbf{l}$ currently tracked in $\mu^\sharp$.

No substitution is returned since field identifiers are pre-determined by the naming convention; no memory reorganization occurs. Note that this case could be further refined with techniques such as [4] to operate directly with the lattice element S without concretizing it.

$\text{assume}_{\mathcal{M}^\sharp}$. Boolean guards constrain values but do not affect the pointer structure:

$$\text{assume}_{\mathcal{M}^\sharp}(\mathbf{b}, \mu^\sharp) = \mu^\sharp.$$

$\text{assign}_{\mathcal{M}^\sharp}$. This is the core operation. It updates the points-to relation based on the syntactic form of the assignment, implementing the Andersen inclusion constraints in a flow-sensitive manner. All updates are *accumulative* (we only add to points-to sets, never remove), ensuring soundness at the cost of possible imprecision. The substitution returned is always $\square$ (no identifier renaming is needed), so the value domain is not disturbed.

Because the naming convention makes $\text{fresh}_{\mathcal{M}^\sharp}^\kappa/\text{fresh}_{\mathcal{M}^\sharp}^\mathcal{H}$ deterministic, $\text{assign}_{\mathcal{M}^\sharp}$ can pre-compute the identifier ι_ℓ that will be allocated for any allocation site ℓ in the rhs, and update the points-to set of the lhs accordingly before $R_{\mathcal{M}^\sharp}$ is invoked.

The main cases are:

- *Address-of* (lhs = $x \in \text{ID}$, rhs = $\&y$ with $y \in \text{ID}$) records the Andersen address-of constraint, that is, that x may point to y 's abstract location ι_y :

$$\text{assign}_{\mathcal{M}^\sharp}(x, \&y, \mu^\sharp) = (\square, \mu^\sharp[x \mapsto \mu^\sharp(x) \cup \{\iota_y\}]).$$

- *Pointer copy* (lhs = $x \in \text{ID}$, rhs = $y \in \text{ID}$) propagates y 's points-to set into x 's; if y is not a pointer, $\mu^\sharp(\iota_y) = \emptyset$ and the update is a no-op:

$$\text{assign}_{\mathcal{M}^\sharp}(x, y, \mu^\sharp) = (\square, \mu^\sharp[x \mapsto \mu^\sharp(x) \cup \mu^\sharp(y)]).$$

- *Pointer load* (lhs = $x \in \text{ID}$, rhs = $*p$, with $p \in \text{ID}$) records that x may point to anything that the targets of p itself point to:

$$\text{assign}_{\mathcal{M}^\sharp}(x, *p, \mu^\sharp) = \left(\square, \mu^\sharp \left[x \mapsto \mu^\sharp(x) \cup \bigcup_{\iota \in \mu^\sharp(p)} \mu^\sharp(\iota) \right] \right).$$

- *Pointer store* (lhs = $*p$, rhs = y , with $p \in \text{ID}$) adds y 's points-to set to all possible targets, since p may alias multiple concrete locations:

$$\text{assign}_{\mathcal{M}^\sharp}(*p, y, \mu^\sharp) = \left(\square, \bigsqcup_{\iota \in \mu^\sharp(p)} \mu^\sharp \left[\iota \mapsto \mu^\sharp(\iota) \cup \mu^\sharp(y) \right] \right).$$

- *Object creation* (lhs = $x \in \text{ID}$, rhs $\in \{\{\mathbf{f}_i:\mathbf{v}_i\}, \text{new } \{\mathbf{f}_i:\mathbf{v}_i\}\}$ at allocation site ℓ) records that x may now point to the freshly allocated object ι_ℓ and initialises its field identifiers:

$$\text{assign}_{\mathcal{M}^\sharp}(x, \text{rhs}, \mu^\sharp) = (\square, \mu^\sharp[x \mapsto \mu^\sharp(x) \cup \{\iota_\ell\}, \iota_\ell \mapsto \emptyset, \iota_{\iota_\ell, \mathbf{f}_i} \mapsto \emptyset \mid i]).$$

- *Field assignment* (lhs = $x.f$ or lhs = $x[s]$, rhs = y) is analogous to the pointer-store case, propagating y 's points-to set into the appropriate field identifiers of x 's targets.
- *Non-pointer assignment* (rhs is a VE or AE expression that cannot yield an address) behaves like a no-op on the memory domain:

$$\text{assign}_{\mathcal{M}^\sharp}(\text{lhs}, \text{rhs}, \mu^\sharp) = (\square, \mu^\sharp).$$

Example 5 (Application to the Running Example). We trace the points-to analysis on the μLL program of Listing 2.

Identifiers. The naming convention introduces one base identifier per variable and one per allocation site:

- $\iota_{\text{list}}, \iota_{\mathbf{x}}, \iota_{\text{tail}}, \iota_{\mathbf{n}}, \iota_{\text{tmp}}$ — variable identifiers;
- ι_{ℓ_1} — stack allocation site for $\text{list} = \{\}$, an empty object: no field identifiers exist until a field is first written;
- ι_{ℓ_7} — heap allocation site for $\text{new}\{\mathbf{L} : \text{tail}\}$ (fixed across loop iterations by allocation-site abstraction).

Field identifiers are written $\iota_{\iota, f}$ for field f of object ι .

After Initialisation (Lines 1–3). Starting from $\mu_0^\# = \emptyset$:

- Line 1 ($\text{list} = \{\}$): $\text{assign}_{\mathcal{M}^\#}$ (object-creation case) extends the state with $\text{list} \mapsto \{\iota_{\ell_1}\}$ and $\iota_{\ell_1} \mapsto \emptyset$. Since the object literal declares no fields, no field identifiers are created yet.
- Line 2 ($\mathbf{x} = \&\text{list}$): $\text{assign}_{\mathcal{M}^\#}$ (address-of case) sets $\mathbf{x} \mapsto \{\iota_{\text{list}}\}$. $\mathbf{R}_{\mathcal{M}^\#}(\&\text{list})$ calls $\text{addrOf}_{\mathcal{M}^\#}(\text{list}, \mu^\#)$ and returns $\{\mathbf{x}, \iota_{\text{list}}\}$ directly.
- Line 3 ($\text{tail} = \&\text{list}$): analogous, giving $\text{tail} \mapsto \{\iota_{\text{list}}\}$.

The state after line 3 is:

$$\mu_3^\# = \{\text{list} \mapsto \{\iota_{\ell_1}\}, \mathbf{x} \mapsto \{\iota_{\text{list}}\}, \text{tail} \mapsto \{\iota_{\text{list}}\}, \iota_{\ell_1} \mapsto \emptyset\}$$

Loop Body (Lines 7–10).

- Line 7 ($\mathbf{n} = \text{new}\{\mathbf{L} : \text{tail}\}$): $\text{assign}_{\mathcal{M}^\#}$ (object-creation) adds $\mathbf{n} \mapsto \{\iota_{\ell_7}\}$ and $\iota_{\ell_7} \mapsto \emptyset$, initialising the field identifier $\iota_{\iota_{\ell_7}, \mathbf{L}} \mapsto \emptyset$ (there is no $\mathbf{R}$ field this time, since the literal only declares $\mathbf{L}$). Then $\text{access}_{\mathcal{M}^\#}(\{\iota_{\ell_7}\}, \mathbf{L}, \mu^\#)$ produces $\iota_{\iota_{\ell_7}, \mathbf{L}}$, and the field-assignment case of $\text{assign}_{\mathcal{M}^\#}$ propagates tail 's points-to set into it:

$$\iota_{\iota_{\ell_7}, \mathbf{L}} \mapsto \mu^\#(\text{tail}) = \{\iota_{\text{list}}\}.$$

- Line 8 ($\text{tmp} = *\text{tail}$): pointer-load case of $\text{assign}_{\mathcal{M}^\#}$:

$$\text{tmp} \mapsto \bigcup_{\iota \in \mu^\#(\text{tail})} \mu^\#(\iota) = \mu^\#(\text{list}) = \{\iota_{\ell_1}\}.$$

After this, tmp and list both point to ι_{ℓ_1} : tmp is an alias of list 's object.

- Line 9 ($\text{tmp}.\mathbf{R} = \mathbf{n}$): field-assignment (weak update). $\text{access}_{\mathcal{M}^\#}$ resolves tmp 's target to ι_{ℓ_1} and selects the field identifier $\iota_{\iota_{\ell_1}, \mathbf{R}}$, not yet tracked at this point since list 's object had no $\mathbf{R}$ field until now. By the domain's convention, a non-tracked identifier has an empty points-to set, so the update introduces it directly with $\mathbf{n}$'s points-to set:

$$\iota_{\iota_{\ell_1}, \mathbf{R}} \mapsto \emptyset \cup \mu^\#(\mathbf{n}) = \{\iota_{\ell_7}\}.$$

This is an *over-approximation*: a strong update would replace the old value, but since $|\gamma_{\text{MemID}}(\iota_{\ell_1})| > 1$ is possible, we conservatively join.

- Line 10 (`*tail = tmp`): pointer-store case. $\mu^\#(\text{tail}) = \{\iota_{\text{list}}\}$, so the weak update adds $\mu^\#(\text{tmp}) = \{\iota_{\ell_1}\}$ to $\mu^\#(\text{list})$, which already contains ι_{ℓ_1} : no change.
- Lines 12–16 (if-branch `x = n` at line 13, else `skip` at line 15): pointer-copy in the true branch gives $\mathbf{x} \mapsto \{\iota_{\text{list}}\} \cup \{\iota_{\ell_7}\}$. After joining the two branches at the loop head (line 17): $\mathbf{x} \mapsto \{\iota_{\text{list}}, \iota_{\ell_7}\}$.

Fixpoint. Re-entering the loop with the end-of-body state, no entry grows further (allocation-site abstraction collapses all heap nodes created at line 7 into the single ι_{ℓ_7} ; all points-to sets are already at their join values). The analysis reaches a fixpoint after one iteration. The final abstract memory state is:

$$\mu^{\#*} = \left\{ \begin{array}{l} \text{list} \mapsto \{\iota_{\ell_1}\}, \iota_{\iota_{\ell_1}, \text{R}} \mapsto \{\iota_{\ell_7}\}, \mathbf{x} \mapsto \{\iota_{\text{list}}, \iota_{\ell_7}\}, \text{tail} \mapsto \{\iota_{\text{list}}\}, \\ \mathbf{n} \mapsto \{\iota_{\ell_7}\}, \iota_{\iota_{\ell_7}, \text{L}} \mapsto \{\iota_{\text{list}}\}, \text{tmp} \mapsto \{\iota_{\ell_1}\}, \dots \end{array} \right\}$$

Soundness. The operations above are sound with respect to $\gamma_{\mathcal{M}^\sharp}$ because each one can only *grow* points-to sets: it adds identifiers to sets but never removes them. Since $\gamma_{\mathcal{M}^\sharp}$ is monotone (Lemma 4), any concrete state that was consistent with the pre-state remains consistent with the post-state. More precisely:

- `addrOf $\mathcal{M}^\sharp$` : returns the set of identifiers referring to the given ι , which by the soundness condition on $\mu^\#$ over-approximates all concrete referrers. The state is unchanged.
- `pointsTo $\mathcal{M}^\sharp$` : returns the set $\mu^\#(\iota)$, which by the soundness condition on $\mu^\#$ over-approximates all concrete targets. The state is unchanged.
- `isSum $\mathcal{M}^\sharp$` : always returns `true`, since all identifiers are summaries. The state is unchanged.
- `fresh $\mathcal{M}^\sharp$ K/fresh $\mathcal{M}^\sharp$ H`: adds $\iota_\ell \mapsto \emptyset$. The empty points-to set is a sound over-approximation of a freshly allocated location (which holds no pointer yet); concrete executions that later store a pointer will be captured by subsequent `assign $\mathcal{M}^\sharp$` calls.
- `access $\mathcal{M}^\sharp$` : adds field identifiers with empty points-to sets; soundness follows as for `fresh $\mathcal{M}^\sharp$ K`.
- `assign $\mathcal{M}^\sharp$` (*pointer copy/load/store/object-creation*): each case adds identifiers to the target’s points-to set, over-approximating the concrete semantics: any concrete address that can reach x after the assignment is covered by the updated $\mu^\#(\iota_x)$. For the weak store case `*p = y`, updating *all* potential targets of p (rather than exactly one) is a sound over-approximation.
- `assume $\mathcal{M}^\sharp$` : the identity is trivially sound since boolean guards do not modify memory structure.

By Theorem 3, these per-operation soundness arguments together guarantee that the combined analysis (interval domain for values, points-to domain for memory) is sound for μLL .

7.3 Combining the Instantiations

To demonstrate how the two instantiations cooperate on a concrete setting, we extend the running example of Listing 2 so that each allocated node also carries an integer field `val`. A counter variable `count` is initialised to 1 before the loop. At line 7, the heap allocation becomes `n = new {L : tail, val : 1}`, and two new statements `n.val = count` followed by `count = count + 1` are inserted at the end of the loop body. In this variant, we describe the cooperation between $\mathcal{M}^\sharp$ and $\mathcal{V}^\sharp$ in a mixed pointer/integer setting, demonstrating how Andersen analysis interacts with non-relational value domains, such as Sign and Interval, within the proposed framework.

Effect on the Memory Domain. The `val` field is integer-typed and holds no pointer.

- Line 7: the object-creation case of `assignM‡` introduces a field identifier $\iota_{\ell_7, \text{val}}$ into $\mu^\sharp$, mapped to $\emptyset$. Its points-to set remains $\emptyset$ throughout the analysis, so the points-to graph is unaffected.
- Counter increment and store (`count = count + 1` then `n.val = count`): both right-hand sides are arithmetic expressions, so the *non-pointer assignment* case of `assignM‡` applies to each. No structural change occurs and no substitution is emitted by either statement.

Consequently, the abstract memory state is identical to that of the original example, and the points-to fixpoint is still reached after one loop iteration.

Effect on the Value Domain and Domain Interaction. The interaction between $\mathcal{M}^\sharp$ and $\mathcal{V}^\sharp$ is mediated by the rewriting function $R_{M^\sharp}$ and the operations `accessM‡` and `assignV‡`, as defined in Section 6. We trace the processing of the two new statements in turn.

Step A: `n.val = count`.

1. *Memory domain first.* `assignM‡` returns $([], \mu^\sharp)$; the empty substitution leaves $\nu^{\sharp'}$ unchanged.
2. *Right-hand-side rewriting.* $R_{M^\sharp}(\text{count}, \nu^\sharp, \mu^\sharp)$ resolves `count` directly (it is a concrete identifier; no `accessM‡` call is needed) and returns its current abstract value from $\nu^\sharp$.
3. *Left-hand-side rewriting.* $R_{M^\sharp}(\text{n.val}, \nu^\sharp, \mu^\sharp)$ calls `accessM‡( $\{\iota_{\ell_7}\}, \text{val}, \mu^\sharp)$` , returning the field identifier $\iota_{\ell_7, \text{val}}$.
4. *Value update.* Since $\text{isSum}_{M^\sharp}(\iota_{\ell_7}, \mu^\sharp) = \text{true}$ (every identifier is a summary node in the Andersen domain), the boolean flag passed to `assignV‡` is `false`, forcing a *weak update* (join with the existing abstract value):

$$\nu^{\sharp'} = \text{assign}_{V^\sharp}(\iota_{\ell_7, \text{val}}, \text{count}, \nu^\sharp, \text{false}).$$

Step B: `count = count + 1`. `count` $\in \text{ID}$ is a concrete program variable, so no `accessM‡` call is needed. $R_{M^\sharp}(\text{count} + 1, \nu^{\sharp'}, \mu^\sharp)$ leaves the expression unchanged

as `count + 1`. Because `count` is not behind any pointer, `isSumM` is not consulted and `assignV` applies a *strong update*:

$$\nu^{\#''} = \text{assign}_{\mathcal{V}^{\#}}(\text{count}, \text{count} + 1, \nu^{\#}, \text{true}).$$

Sign Domain. Instantiating $\mathcal{V}^{\#}$ with the sign domain, the abstract sign of `count` is $+$ (strictly positive) from initialisation, since `count` = 1 > 0. After the strong update `count` = `count` + 1, the sign remains $+$: adding one to a positive integer is still positive. At the loop head, joining $+$ with $+$ gives $+$; the ascending chain is already stable after the first iteration and no widening is required. The weak update `n.val` = `count` propagates the sign into the field identifier:

$$\iota_{\ell_7, \text{val}} \mapsto \underbrace{+}_{\text{creation}} \sqcup_{\mathcal{V}^{\#}} + = +.$$

The combined state $(\nu^{\#}, \mu^{\#}) \in \mathcal{S}^{\#}$ reaches a fixpoint after one loop iteration.

Interval Domain. Instantiating $\mathcal{V}^{\#}$ with the interval domain, the abstract value of `count` evolves across loop iterations via *strong* updates:

$$\underbrace{[1, 1]}_{\text{init}} \xrightarrow{+1} [2, 2] \xrightarrow{+1} [3, 3] \rightarrow \dots$$

This ascending chain (the pre-loop value $[1, 1]$ is joined with the post-body value at the loop head) does not stabilise without widening. Applying $\nabla_{\mathcal{V}^{\#}}$ at the loop head gives:

$$[1, 1] \nabla_{\mathcal{V}^{\#}} [2, 2] = [1, +\infty].$$

With `count` stabilised at $[1, +\infty)$, the weak update `n.val` = `count` yields:

$$\iota_{\ell_7, \text{val}} \mapsto \underbrace{[1, 1]}_{\text{creation}} \sqcup_{\mathcal{V}^{\#}} [1, +\infty] = [1, +\infty],$$

after which no further growth occurs and the combined state $(\nu^{\#}, \mu^{\#}) \in \mathcal{S}^{\#}$ reaches a fixpoint.

Summary of Domain Cooperation. The two instantiations above highlight the modularity of the framework. In both cases $\mu^{\#}$ stabilises after one loop iteration. The value domain $\mathcal{V}^{\#}$, however, behaves differently: the sign domain converges immediately (no widening required), while the interval domain produces a diverging chain and requires one widening step to stabilise `count` to $[1, +\infty]$, after which $\iota_{\ell_7, \text{val}}$ follows to $[1, +\infty]$ via the weak update. This example also illustrates the distinction between *strong* and *weak* updates: `count`, a concrete program variable, receives a strong (overwriting) update, whereas $\iota_{\ell_7, \text{val}}$, a field of a summary node, receives a weak (joining) update because `isSumM`($\iota_{\ell_7}, \mu^{\#}$) = `true`. Crucially, neither component waits for the other: the memory domain reaches its fixpoint regardless of what the value domain does

with `count` or `val`, and the value domain widens (or not) without triggering any structural change in $\mu^\sharp$. This independent convergence is precisely the modular design the $\mathcal{V}^\sharp \otimes \mathcal{M}^\sharp$ construction is intended to exploit. The two components interact exclusively through the shared vocabulary of memory identifiers: the memory domain creates the field identifier $\iota_{\ell_7, \text{val}}$ via `accessMz`, and signals through `isSumMz` that the corresponding node is a summary, thereby forcing a weak update in $\mathcal{V}^\sharp$. The memory domain never inspects interval values; the value domain never inspects the points-to graph. This separation is the defining property of the smash-product construction $\mathcal{S}^\sharp = \mathcal{V}^\sharp \otimes \mathcal{M}^\sharp$ of Section 6: each domain is responsible for its own fragment of the program state, and cooperation reduces to exchanging memory identifiers and substitutions.

8 Conclusions

Stack and heap management is challenging in program static analysis. Precise reasoning about dynamic and static memory allocations and their interactions remains a fundamental difficulty in designing scalable and accurate analyses. In this paper, we propose a generic framework for handling heap and stack that is agnostic to both memory and value analyses. The framework is parametric, allowing different abstractions to be instantiated and composed depending on the precision and performance requirements of the analysis. By clearly separating concerns and enabling modular combination of memory and value reasoning components, our approach provides a flexible foundation for building static analyses that can be adapted to different programming languages and analysis domains.

In future work, we plan to integrate this framework into a general-purpose static analysis platform supporting multiple programming languages, such as LiSA (Library for Static Analysis) [37,21,36]. The framework of [20] is already implemented therein, providing modular interfaces for advanced memory and value analyses. The code⁵ can be used as a starting point for an implementation of this work. Once such development is completed, we aim at providing support to C, C++, Rust, and other low-level languages such as LLVM-IR, in LiSA. Furthermore, we aim at integrating support for pointer arithmetics in the framework by exploiting `evalVz`.

Acknowledgments. This work has been partially supported by Regione Veneto: RIR project SATCO (CUP D19J24000750007), and RIR project SUPREME (CUP D19J24000810007), and by PR Veneto FESR 2021-2027, Az. 1.4.1 ID 251382_008101 “SmartCycle – Cybersecurity”, CUP: H73C25001340002.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

⁵ <https://github.com/lisa-analyzer/lisa/blob/master/lisa/lisa-sdk/src/main/java/it/unive/lisa/analysis/SimpleAbstractDomain.java>

References

1. Andersen, L.O.: Program Analysis and Specialization for the C Programming Language. Ph.D. thesis, DIKU, University of Copenhagen (May 1994), <https://www.cs.aau.dk/~amoeller/papers/andersen/andersen94.pdf>
2. Arceri, V., Maffeis, S.: Abstract domains for type juggling. In: Mastroeni, I. (ed.) Proceedings of the Sixth Workshop on Numerical and Symbolic Abstract Domains, NSADSAS 2016, Edinburgh, UK, September 11, 2016. Electronic Notes in Theoretical Computer Science, vol. 331, pp. 41–55. Elsevier (2016). <https://doi.org/10.1016/J.ENTCS.2017.02.003>, <https://doi.org/10.1016/j.entcs.2017.02.003>
3. Arceri, V., Mastroeni, I.: Static program analysis for string manipulation languages. Electronic Proceedings in Theoretical Computer Science **299**, 19–33 (Aug 2019). <https://doi.org/10.4204/eptcs.299.5>, <http://dx.doi.org/10.4204/EPTCS.299.5>
4. Arceri, V., Pasqua, M., Mastroeni, I.: An abstract domain for objects in dynamic programming languages. In: Sekerinski, E., Moreira, N., Oliveira, J.N., Ratiu, D., Guidotti, R., Farrell, M., Luckcuck, M., Marmsoler, D., Campos, J., Astarte, T., Gonnord, L., Cerone, A., Couto, L., Dongol, B., Kutrib, M., Monteiro, P., Delmas, D. (eds.) Formal Methods. FM 2019 International Workshops. pp. 136–151. Springer International Publishing, Cham (2020)
5. Balakrishnan, G., Reps, T.: Recency-abstraction for heap-allocated storage. In: Yi, K. (ed.) Static Analysis. pp. 221–239. Springer Berlin Heidelberg, Berlin, Heidelberg (2006)
6. Blanchet, B., Cousot, P., Cousot, R., Feret, J., Mauborgne, L., Miné, A., Monniaux, D., Rival, X.: A static analyzer for large safety-critical software. SIGPLAN Not. **38**(5), 196–207 (May 2003). <https://doi.org/10.1145/780822.781153>
7. Blazy, S., Laporte, V., Maroneze, A., Pichardie, D.: Formal verification of a c value analysis based on abstract interpretation. In: International Static Analysis Symposium. pp. 324–344. Springer (2013)
8. Blazy, S., Laporte, V., Pichardie, D.: An abstract memory functor for verified c static analyzers. In: Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming. pp. 325–337 (2016)
9. Boldini, G., Negrini, L., Olivieri, L., Ferrara, P.: A modular framework for stack-heap and value abstractions (extended version) (2026), <https://arxiv.org/abs/2607.15932>
10. Calzavara, S., Grishchenko, I., Koutsos, A., Maffei, M.: A sound flow-sensitive heap abstraction for the static analysis of android applications. In: 2017 IEEE 30th Computer Security Foundations Symposium (CSF). pp. 22–36 (2017). <https://doi.org/10.1109/CSF.2017.19>
11. Chang, B.E., Rival, X.: Relational inductive shape analysis. In: Necula, G.C., Wadler, P. (eds.) Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2008, San Francisco, California, USA, January 7–12, 2008. pp. 247–260. ACM (2008). <https://doi.org/10.1145/1328438.1328469>, <https://doi.org/10.1145/1328438.1328469>
12. Chang, B.E., Rival, X.: Modular construction of shape-numeric analyzers. In: Banerjee, A., Danvy, O., Doh, K., Hatchiff, J. (eds.) Semantics, Abstract Interpretation, and Reasoning about Programs: Essays Dedicated to David A. Schmidt on the Occasion of his Sixtieth Birthday, Manhattan, Kansas, USA, 19–20th September 2013. pp. 161–185. EPTCS (2013). <https://doi.org/10.4204/EPTCS.129.11>, <https://doi.org/10.4204/EPTCS.129.11>

13. Cortesi, A., Costantini, G., Ferrara, P.: A survey on product operators in abstract interpretation. In: Banerjee, A., Danvy, O., Doh, K., Hatcliff, J. (eds.) *Semantics, Abstract Interpretation, and Reasoning about Programs: Essays Dedicated to David A. Schmidt on the Occasion of his Sixtieth Birthday*, Manhattan, Kansas, USA, 19-20th September 2013. EPTCS, vol. 129, pp. 325–336 (2013). <https://doi.org/10.4204/EPTCS.129.19>, <https://doi.org/10.4204/EPTCS.129.19>
14. Cousot, P., Cousot, R.: Systematic design of program analysis frameworks. In: *Conference Record of the Sixth Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. pp. 269–282. ACM Press, New York, NY, San Antonio, Texas (1979)
15. Cousot, P.: *Principles of Abstract Interpretation*. MIT Press (2021)
16. Cousot, P., Cousot, R.: Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In: Graham, R.M., Harrison, M.A., Sethi, R. (eds.) *Conference Record of the Fourth ACM Symposium on Principles of Programming Languages*, Los Angeles, California, USA, January 1977. pp. 238–252. ACM (1977). <https://doi.org/10.1145/512950.512973>, <https://doi.org/10.1145/512950.512973>
17. Cousot, P., Halbwachs, N.: Automatic discovery of linear restraints among variables of a program. In: *Proceedings of the 5th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*. p. 84–96. POPL '78, Association for Computing Machinery, New York, NY, USA (1978). <https://doi.org/10.1145/512760.512770>, <https://doi.org/10.1145/512760.512770>
18. Fahndrich, M., Logozzo, F.: Static contract checking with abstract interpretation. In: Beckert, B., Marché, C. (eds.) *Formal Verification of Object-Oriented Software*. pp. 10–30. Springer Berlin Heidelberg, Berlin, Heidelberg (2011)
19. Ferrara, P.: Generic combination of heap and value analyses in abstract interpretation. In: *International Conference on Verification, Model Checking, and Abstract Interpretation*. pp. 302–321. Springer (2014). https://doi.org/10.1007/978-3-642-54013-4_17
20. Ferrara, P.: A generic framework for heap and value analyses of object-oriented programming languages. *Theor. Comput. Sci.* **631**, 43–72 (2016). <https://doi.org/10.1016/J.TCS.2016.04.001>, <https://doi.org/10.1016/j.tcs.2016.04.001>
21. Ferrara, P., Negrini, L., Arceri, V., Cortesi, A.: Static analysis for dummies: experiencing lisa. In: *Proceedings of the 10th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis*. p. 1–6. SOAP 2021, Association for Computing Machinery, New York, NY, USA (2021). <https://doi.org/10.1145/3460946.3464316>, <https://doi.org/10.1145/3460946.3464316>
22. Gopan, D., DiMaio, F., Dor, N., Reps, T.W., Sagiv, S.: Numeric domains with summarized dimensions. In: Jensen, K., Podelski, A. (eds.) *Tools and Algorithms for the Construction and Analysis of Systems*, 10th International Conference, TACAS 2004, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2004, Barcelona, Spain, March 29 - April 2, 2004, *Proceedings*. pp. 512–529. *Lecture Notes in Computer Science*, Springer (2004). https://doi.org/10.1007/978-3-540-24730-2_38, https://doi.org/10.1007/978-3-540-24730-2_38
23. Gopan, D., Reps, T.W., Sagiv, S.: A framework for numeric analysis of array operations. In: Palsberg, J., Abadi, M. (eds.) *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL 2005, Long Beach, California, USA, January 12-14, 2005. pp. 338–350.

- ACM (2005). <https://doi.org/10.1145/1040305.1040333>, <https://doi.org/10.1145/1040305.1040333>
24. Granger, P.: Static analysis of arithmetical congruences. *International Journal of Computer Mathematics* **30**(3-4), 165–190 (1989)
 25. Hind, M.: Pointer analysis: haven’t we solved this problem yet? In: *Proceedings of the 2001 ACM SIGPLAN-SIGSOFT Workshop on Program Analysis for Software Tools and Engineering*. p. 54–61. PASTE ’01, Association for Computing Machinery, New York, NY, USA (2001). <https://doi.org/10.1145/379605.379665>
 26. Journault, M., Miné, A., Monat, R., Ouadjaout, A.: Combinations of reusable abstract domains for a multilingual static analyzer. In: *Working Conference on Verified Software: Theories, Tools, and Experiments*. pp. 1–18. Springer (2019)
 27. Kanvar, V., Khedker, U.P.: Heap abstractions for static analysis. *ACM Comput. Surv.* **49**(2) (Jun 2016). <https://doi.org/10.1145/2931098>, <https://doi.org/10.1145/2931098>
 28. Lev-Ami, T., Sagiv, M.: Tvla: A system for implementing static analyses. In: *International Static Analysis Symposium*. pp. 280–301. Springer (2000)
 29. Milanese, M., Miné, A.: Under-approximating memory abstractions. In: *International Static Analysis Symposium*. pp. 300–326. Springer (2024)
 30. Miné, A.: Field-sensitive value analysis of embedded c programs with union types and pointer arithmetics. In: *Proceedings of the 2006 ACM SIGPLAN/SIGBED Conference on Language, Compilers, and Tool Support for Embedded Systems*. p. 54–63. LCTES ’06, Association for Computing Machinery, New York, NY, USA (2006). <https://doi.org/10.1145/1134650.1134659>, <https://doi.org/10.1145/1134650.1134659>
 31. Miné, A.: The octagon abstract domain. *Higher Order Symbol. Comput.* **19**(1), 31–100 (Mar 2006). <https://doi.org/10.1007/s10990-006-8609-1>
 32. Miné, A.: Tutorial on static inference of numeric invariants by abstract interpretation. *Found. Trends Program. Lang.* **4**(3-4), 120–372 (2017). <https://doi.org/10.1561/25000000034>
 33. Monat, R., Milanese, M., Parolini, F., Boillot, J., Ouadjaout, A., Miné, A.: Mopsac: Improved verification for c programs, simple validation of correctness witnesses (competition contribution). In: Finkbeiner, B., Kovács, L. (eds.) *Tools and Algorithms for the Construction and Analysis of Systems*. pp. 387–392. Springer Nature Switzerland, Cham (2024)
 34. Monat, R., Ouadjaout, A., Miné, A.: Value and allocation sensitivity in static python analyses. In: *Proceedings of the 9th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis*. p. 8–13. SOAP 2020, Association for Computing Machinery, New York, NY, USA (2020). <https://doi.org/10.1145/3394451.3397205>, <https://doi.org/10.1145/3394451.3397205>
 35. Negrini, L.: Whole-value analysis by abstract interpretation. *Frontiers in Computer Science* **Volume 7 - 2025** (2026). <https://doi.org/10.3389/fcomp.2025.1655377>, <https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2025.1655377>
 36. Negrini, L., Arceri, V., Olivieri, L., Cortesi, A., Ferrara, P.: Teaching through practice: Advanced static analysis with lisa. In: Sekerinski, E., Ribeiro, L. (eds.) *Formal Methods Teaching*. pp. 43–57. Springer Nature Switzerland, Cham (2024)
 37. Negrini, L., Ferrara, P., Arceri, V., Cortesi, A.: LiSA: A Generic Framework for Multilanguage Static Analysis, pp. 19–42. Springer Nature Singapore, Singapore (2023). https://doi.org/10.1007/978-981-19-9601-6_2, https://doi.org/10.1007/978-981-19-9601-6_2

- 38. Sagiv, M., Reps, T., Wilhelm, R.: Parametric shape analysis via 3-valued logic. *ACM Trans. Program. Lang. Syst.* **24**(3), 217–298 (May 2002). <https://doi.org/10.1145/514188.514190>, <https://doi.org/10.1145/514188.514190>
- 39. Venet, A.: Towards the Integration of Symbolic and Numerical Static Analysis, pp. 227–236. Springer Berlin Heidelberg, Berlin, Heidelberg (2008). https://doi.org/10.1007/978-3-540-69149-5_24